



UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA



CORSO DI GRAFICA E INTERATTIVITA'

2024/2025

Unity Scripting

Lezione 3: Hands on gaming behaviour

Docenti:

Prof. Andrea F. Abate

Dott.ssa Lucia Cimmino



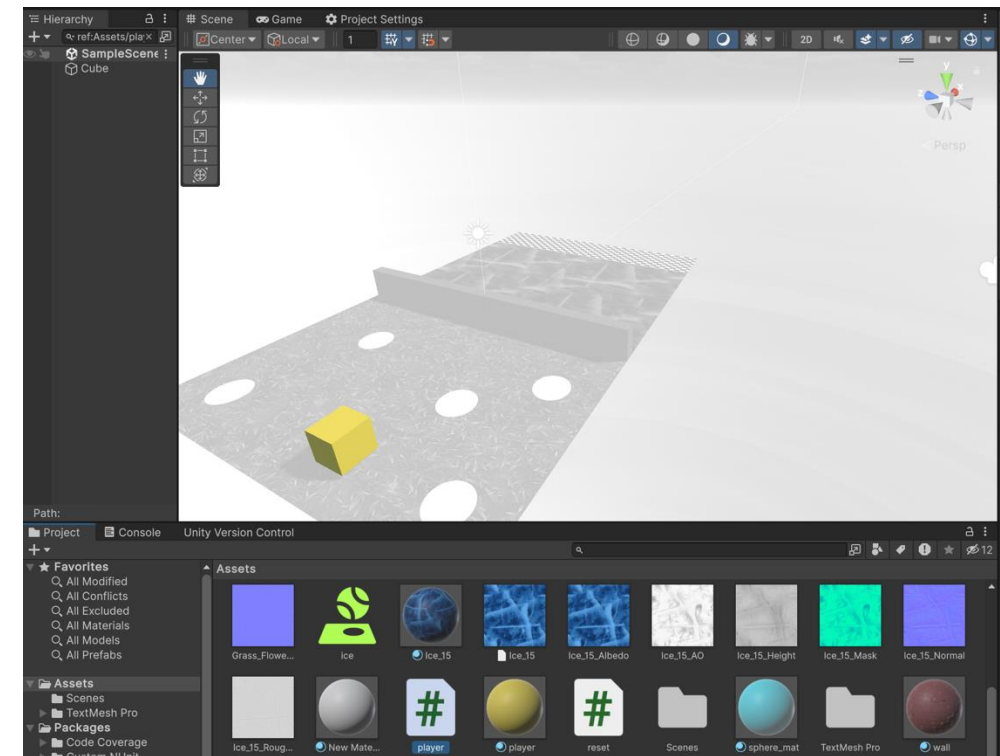
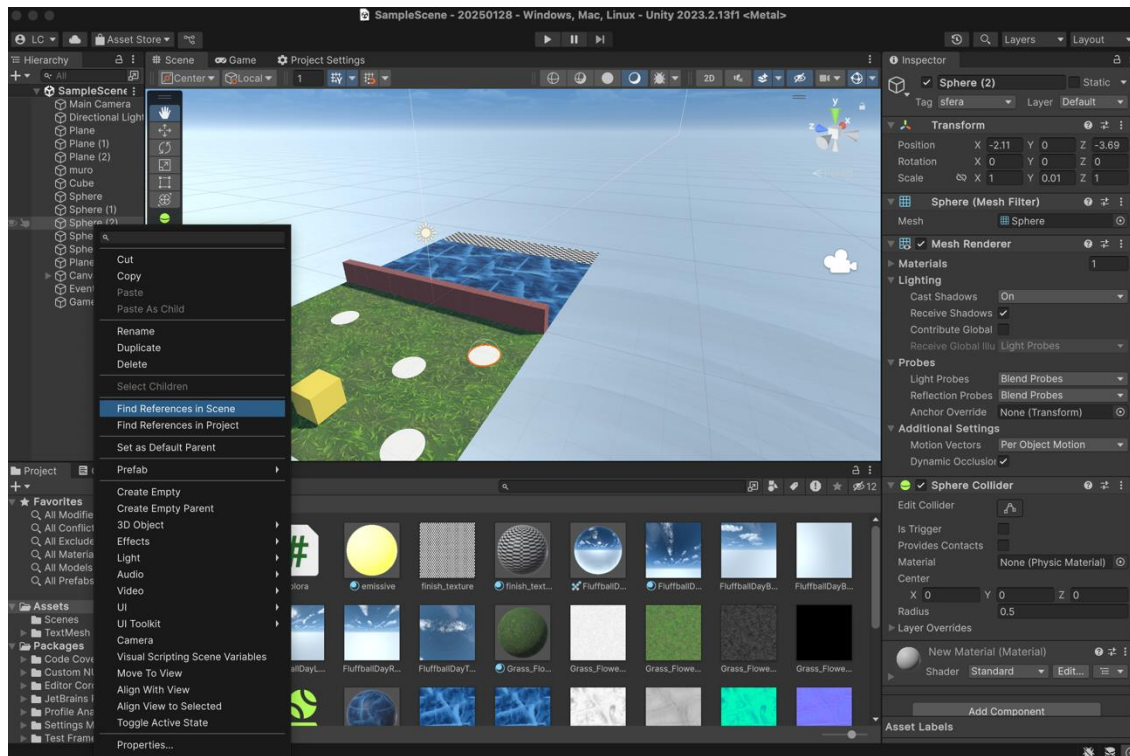
Before Starting...



Find reference in the scene

Find Reference In Scene

Useful command to **locate** where a **specific asset or object** is used within the scene.



Scripting



The basis

Scripting in Unity

Unity Scripting System:

- Unity uses a **custom implementation** of the **Mono runtime** for scripting (backend).
- Scripts are used to **control objects** created in the Unity Editor and the game play.

Current Language: C#

- A **modern object-oriented programming language** developed by Microsoft.
- Part of the **.NET framework**, widely used in game development.

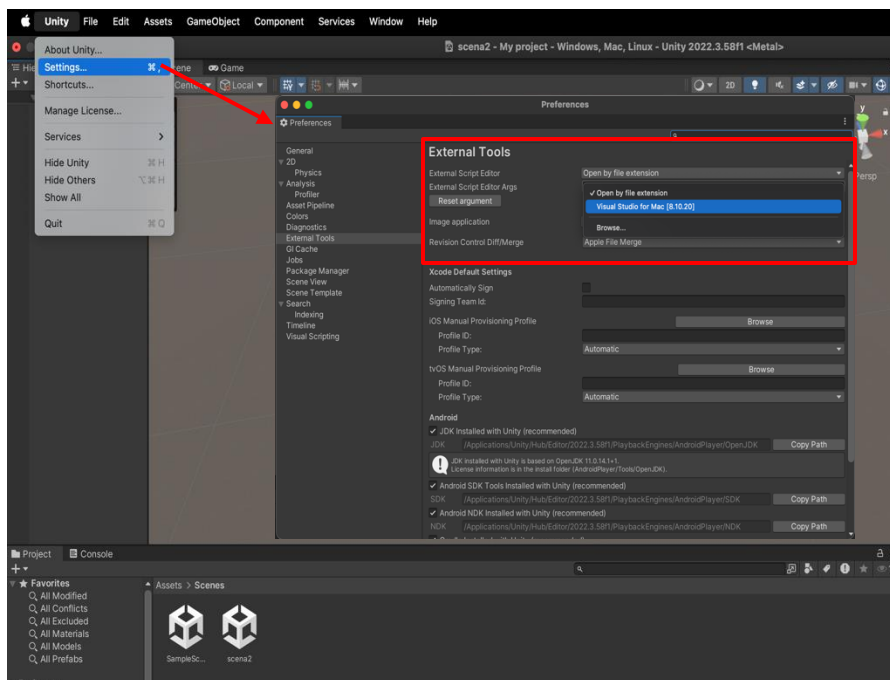
• Previously Supported Languages

- JavaScript (UnityScript): A simplified subset of JavaScript.
- Boo: A statically-typed, Python-inspired object-oriented language.

Since 2017, **C# is the only officially supported language** in Unity.

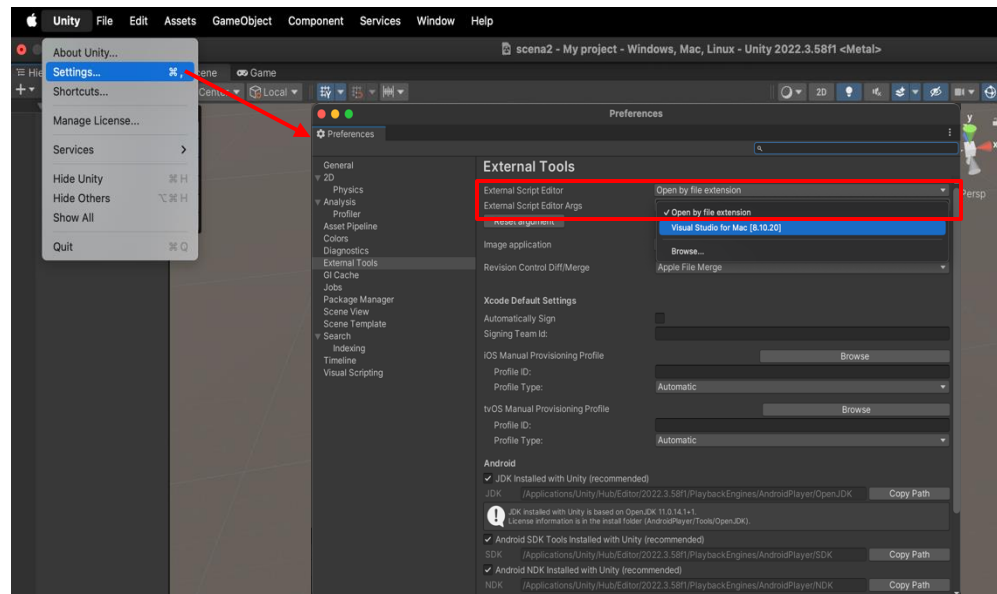
Unity Scripting: IDE

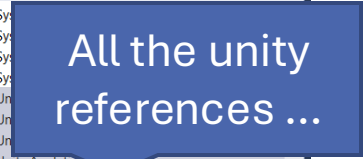
- You can set your **preferred script editor in Unity** → **Settings** → **External Tools**.
- Unity allows integration with popular IDEs such as **Visual Studio**, **Atom**, **Visual Studio Code** and others...



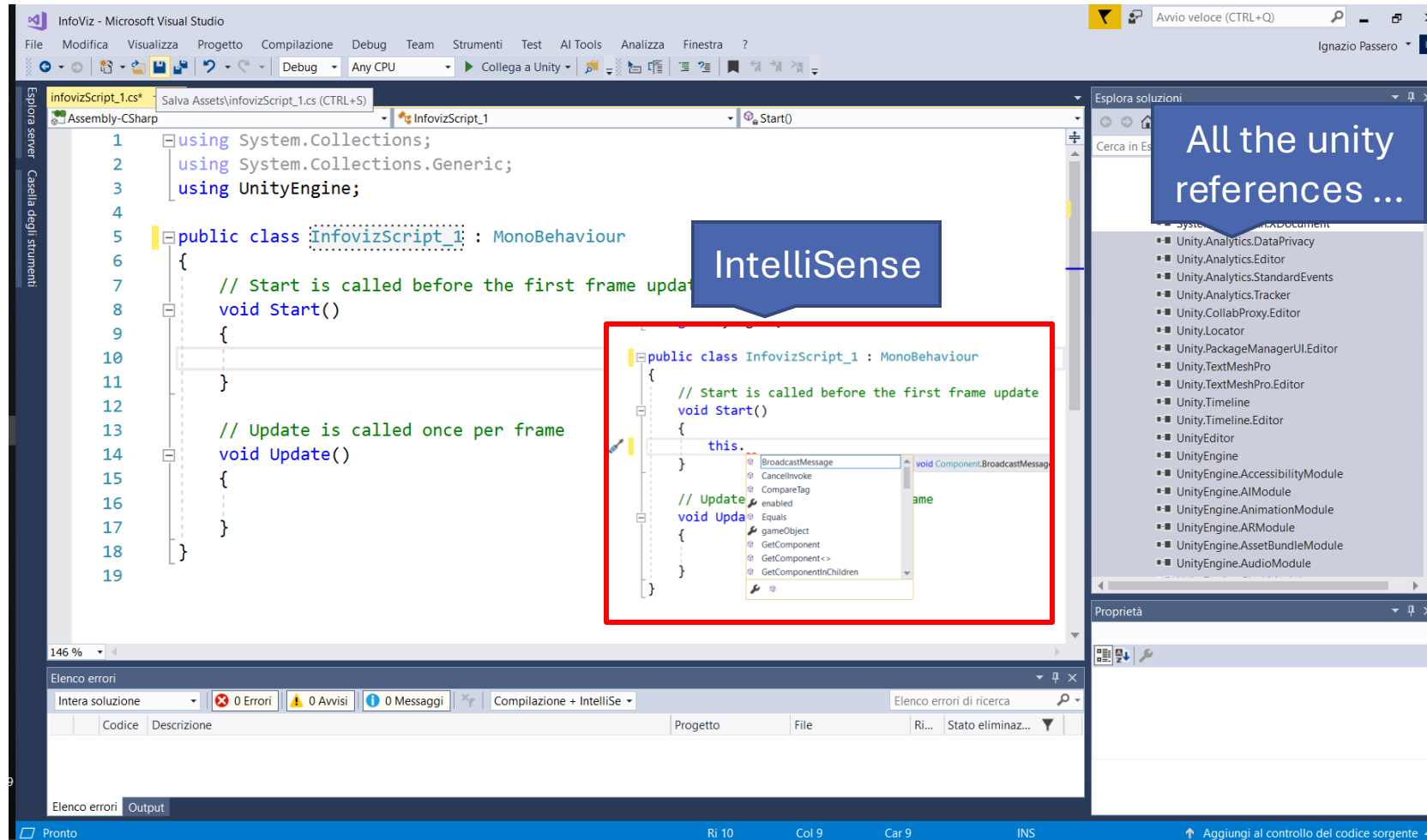
Unity Scripting: IDE

- By default, Unity uses **Visual Studio** as its script editor if it is installed.
- If no external editor is installed, Unity will default to its **built-in text editor**, which has limited functionality.





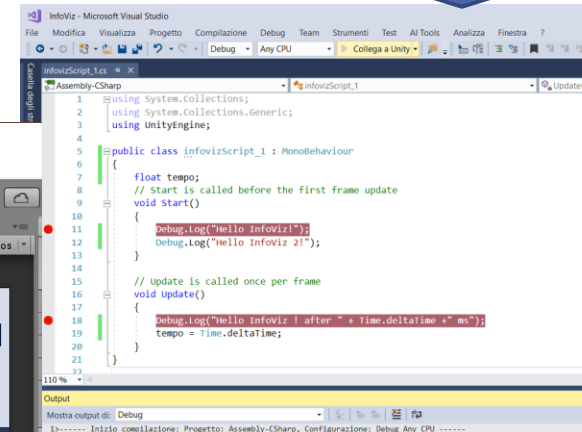
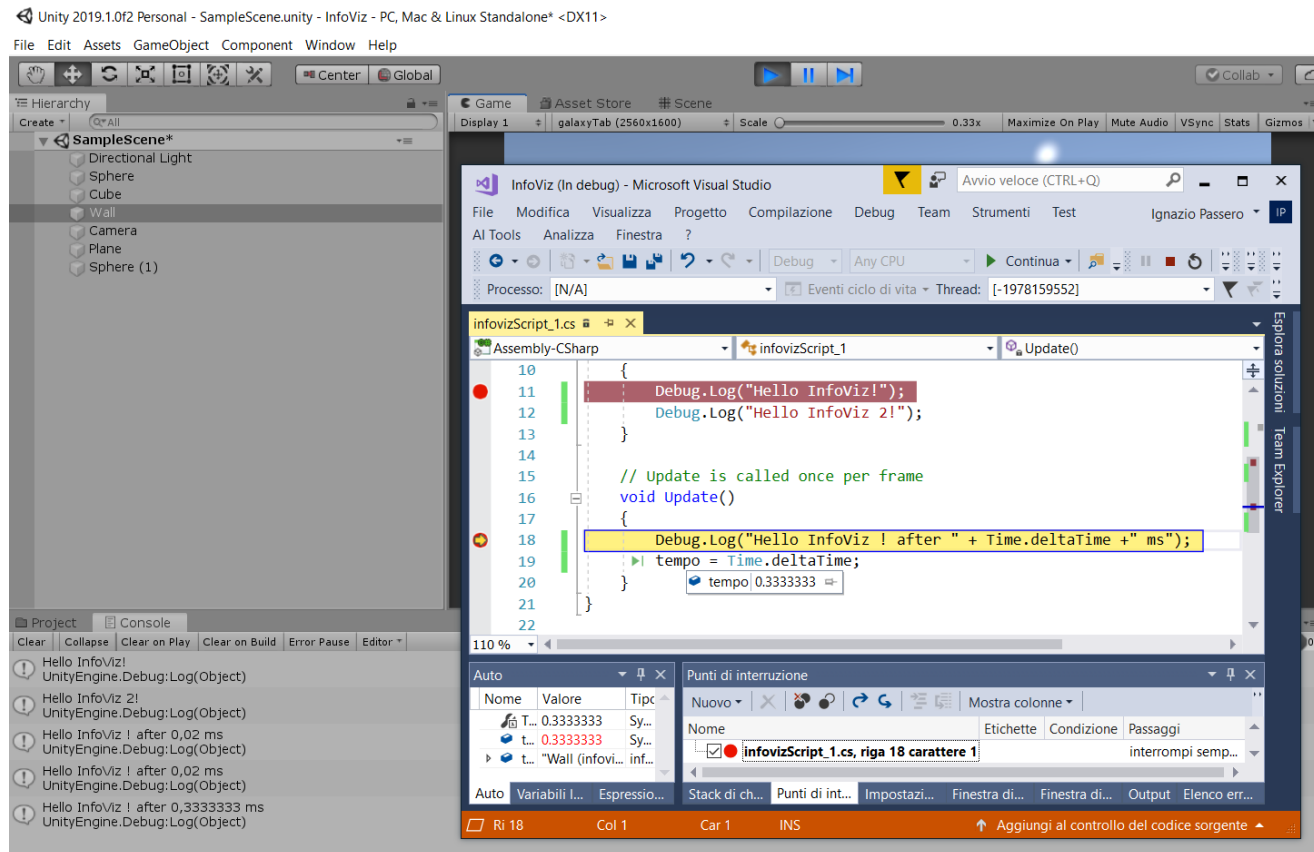
Visual Studio



Visual Studio

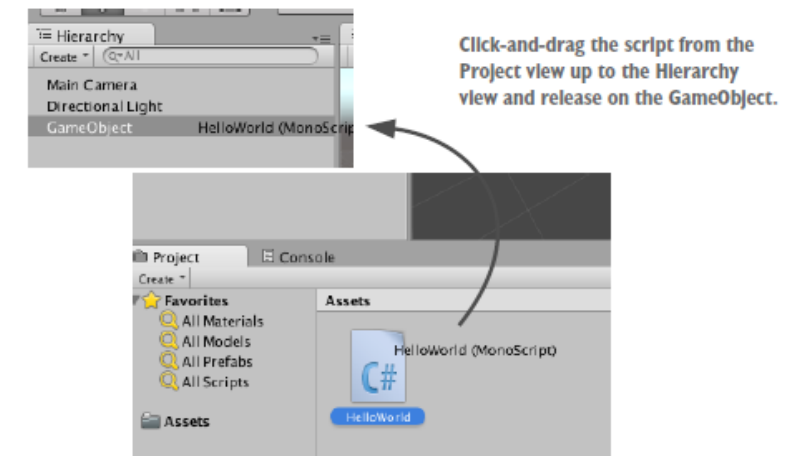


Unity Debug



The Script Component

- In Unity, **scripts** are **components**, meaning they **must** be **attached to a GameObject** to function.
- **Creating a New Script:**
 1. In the **Project** window, go to **Assets** → **Create** → **C# Script**.
 2. Name the script appropriately (e.g., **PlayerController**).
 3. Double-click the script to open it in **Visual Studio** for editing.
- **Attaching a Script to a GameObject:**
 - Drag the script from the **Project view** onto the desired **GameObject** in the **Hierarchy**.



Scripts control **GameObjects' behavior**, just like other components (e.g., Rigidbody, Collider).

A typical C# “empty” Script

```
using UnityEngine;
using System.Collections;

public class ScriptName : MonoBehaviour
{
    // Use this for initialization
    void Start () {
        initializations
    }

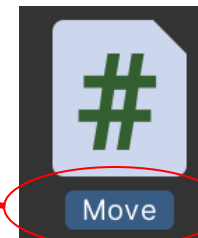
    // Update is called once per frame
    void Update () {
        do something in the current frame
    }
}
```

Names consistency

```
using UnityEngine;
using System.Collections;

public class Move : MonoBehaviour
{
    // Use this for initialization
    void Start () {
        initializations
    }

    // Update is called once per frame
    void Update () {
        do something in the current frame
    }
}
```



!! Keep the class and the script name Consistent !!

When you rename a script, you need to manually update the class name in the script to match

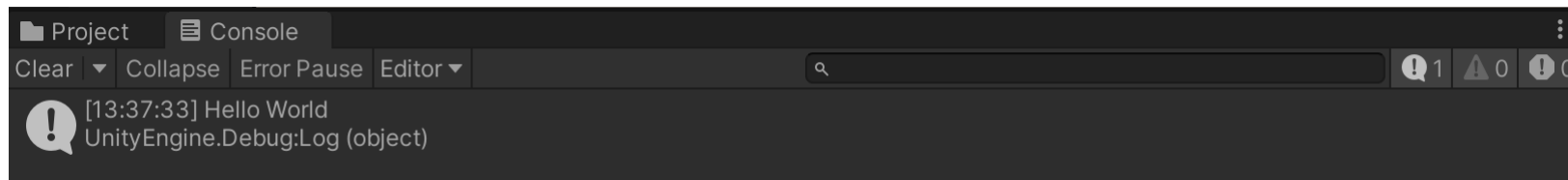
Let's start coding

Listing 1.2 Adding a console message

```
void Start() {  
    Debug.Log("Hello World!");  
}
```

← Add the logging command here.

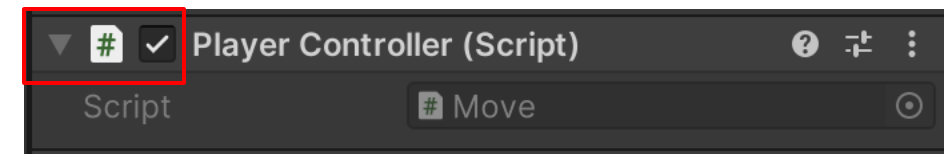
- The **Start()** method is called **once** when you press **Play** in the Unity editor.
- After adding a **log command** to your script, **save** the script before running the scene.
- Press **Play**, then switch to the **Console** window to see the message "**Hello World!**" appear.
- The **Console** is useful for debugging and tracking script execution in Unity.



A more complete C# Script

- **MonoBehaviour** is the **base class** from which all Unity scripts derive.
- When coding in **C#**, you must **explicitly inherit** from MonoBehaviour to access built-in lifecycle methods.
- Unity provides a **checkbox** in the **Inspector** to **enable or disable** a MonoBehaviour script. When **unchecked**, the script's functions are **not executed**, effectively disabling the component.
- However, if none of the following lifecycle methods are present in the script, **the checkbox does not appear** in the Inspector:

Start()
Update()
FixedUpdate()
LateUpdate()
OnGUI()
OnDisable()
OnEnable()



A more complete C# Script

```
using UnityEngine;
using System.Collections;

public class MainPlayer : MonoBehaviour
{
    // Called one on script loading
    void Awake() {
        initializations
    }
    // Use this for initialization
    void Start () {
        initializations
    }

    // Update is called once per frame
    void Update () {
        do something in the current frame
    }
    // FixedUpdate is called after each Physics update
    void FixedUpdate () {
        do something after Physics update
    }
    // LateUpdate is called after Update and
    // FixedUpdate
    void LateUpdate () {
        do something after Update/FixedUpdate
    }
}
```


Awake() and Start()

- Both methods are used for **initialization**, but they have key differences in when they are called.
- **Awake()**
 - Called **when the script is loaded**, before the scene starts running.
 - Executes **even if the script component is disabled** in the Inspector.
- **Start()**
 - Called **just before the first frame update**.
 - Executes **only if the script component is enabled** in the Inspector.

Key Difference: Awake() runs as soon as the script loads, even if disabled, while Start() only runs if the script is enabled.

Update(), FixedUpdate() and LateUpdate()

- These methods are used for **continuous execution** during gameplay, but each serves a different purpose.
- ***Update()*** is called every frame, if MonoBehaviour is enabled.
- ***FixedUpdate*** is called after each Physics update.
- ***LateUpdate*** is called after Update function every frame, if MonoBehaviour is enabled.

Update(), FixedUpdate() and LateUpdate()

- *Update()*

- Called **once per frame**, as long as the **MonoBehaviour** is enabled.
- Used for **game logic, user input, and animations**.
- Frame rate dependent → execution time **varies based on FPS**.

Update(), FixedUpdate() and LateUpdate()

- **FixedUpdate()**

- Called **at a fixed time interval, independent of frame rate.**
- Used for **physics calculations** (e.g., Rigidbody movement).
- May be called **multiple times per frame** if the frame rate is low, ensuring consistent physics behaviour.

Update(), FixedUpdate() and LateUpdate()

- **LateUpdate()**
 - Called **after Update()** each frame, if the **MonoBehaviour is enabled**.
 - Used for **camera movements** or actions that should occur **after all Update logic is processed**.
 - Ensures smooth following behaviour when tracking objects.

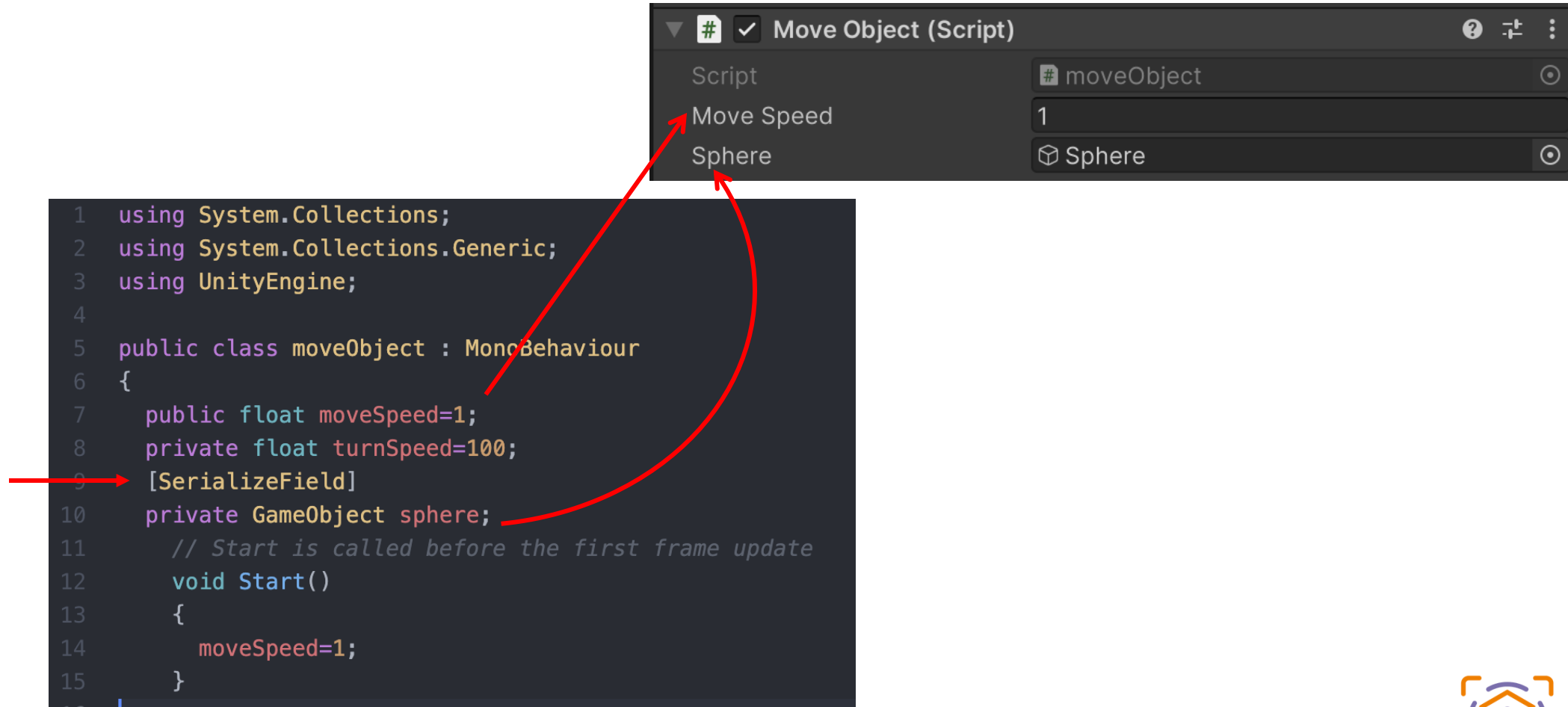
Update(), FixedUpdate() and LateUpdate(): in Summary

Method	When is it called?	Best Used For
Update()	Every frame (frame rate dependent)	Game logic, input handling, animations
FixedUpdate()	At a fixed time step (physics engine update)	Physics calculations, Rigidbody movement
LateUpdate()	After Update() (every frame)	Camera following, final adjustments

Variables and the inspector

- When creating a script, you are essentially creating ***your own new type of component*** that can be attached to Game Objects just like any other component.
- Variables can be:
 - **Public** – also editable outside the class in the Inspector Editor (very useful)
 - **Private** – editable in the script only
- To make a **private variable visible in the Inspector** without making it publicly accessible in other scripts, we can use the attribute `[SerializeField]`

Private and Public variables



The image shows a Unity Inspector window for a script named "Move Object (Script)". The Inspector displays three public variables: "Script" (set to "# moveObject"), "Move Speed" (set to "1"), and "Sphere" (set to "Sphere"). A red arrow points from the "Move Speed" variable in the Inspector to the "moveSpeed" property in the script code. Another red arrow points from the "Sphere" variable in the Inspector to the "sphere" property in the script code. The script code is as follows:

```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 public class moveObject : MonoBehaviour
6 {
7     public float moveSpeed=1;
8     private float turnSpeed=100;
9     [SerializeField]
10    private GameObject sphere;
11    // Start is called before the first frame update
12    void Start()
13    {
14        moveSpeed=1;
15    }
16 }
```


Attributes

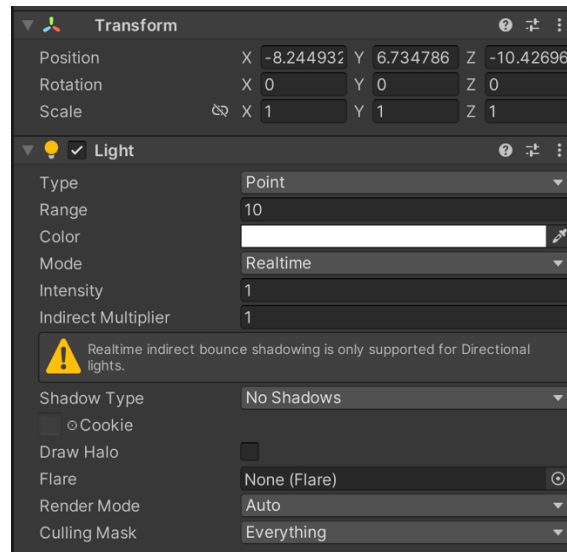
- **Attributes** are special markers placed above classes, methods, or variables to specify **custom behaviour** or **metadata**.
- In C#, attributes are written inside square brackets
- For example, the **[HideInInspector]** attribute can be added above a property declaration to prevent the property being shown in the inspector, even if it is public.

```
// C#  
  
[HideInInspector]  
public float strength;
```

Accessing object components

- In Unity, **GameObjects** are made of **components** which define the object's **behavior and properties**.
- For sure, every game object has a *Transform* component used to move, rotate and scale the object in the scene.
- It is possible to access any component attached to a GameObject using:

```
public Component GetComponent(Type type);
```



```
...  
private Light myLight;  
  
// Use this for initialization  
void Start () {  
    ...  
    myLight = GetComponent<Light>();  
    myLight.color = Color.blue;  
    myLight.intensity = 2;  
    ...  
}  
...
```

Accessing to Object Components (2)

You can access to a Component of type T by using casting:

```
T obj = (T)GetComponent(typeof(T))
```

Example:

```
Light myLight = (Light)GetComponent(typeof(Light));
```

However, by using **generic functions** this task is rather simpler:

```
Type obj = GetComponent<T>();
```

Example:

```
Light myLight = GetComponent<Light>();
```

Generic function makes the programming faster and improves the readability of the whole code.

Accessing to Script Components

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class infovizScript_1 : MonoBehaviour
{
    void Awake()
    {
        infovizScript_1 Script= GetComponent<infovizScript_1>( );
        Script.enabled = true;
    }
}
```

Vector3 Class

- **Vector3** is a struct used to represent **3D vectors and points in space**.
- Commonly used for **position**, **direction**, **velocity**, and **scale** in 3D space.
- Constructor (Creates a new vector with given x, y, z components):

```
public Vector3(float x, float y, float z);
```

- Creates a new vector with given x, y components and sets **z** to zero (by default).

```
public Vector3(float x, float y);
```

Vector3 Class

back

Shorthand for writing `Vector3(0, 0, -1)`.

down

Shorthand for writing `Vector3(0, -1, 0)`.

forward

Shorthand for writing `Vector3(0, 0, 1)`.

left

Shorthand for writing `Vector3(-1, 0, 0)`.

one

Shorthand for writing `Vector3(1, 1, 1)`.

right

Shorthand for writing `Vector3(1, 0, 0)`.

up

Shorthand for writing `Vector3(0, 1, 0)`.

zero

Shorthand for writing `Vector3(0, 0, 0)`.

Create and Destroy objects

Unity provides the **Instantiate()** method to create new **GameObjects** at **runtime** based on a prefab or existing object.

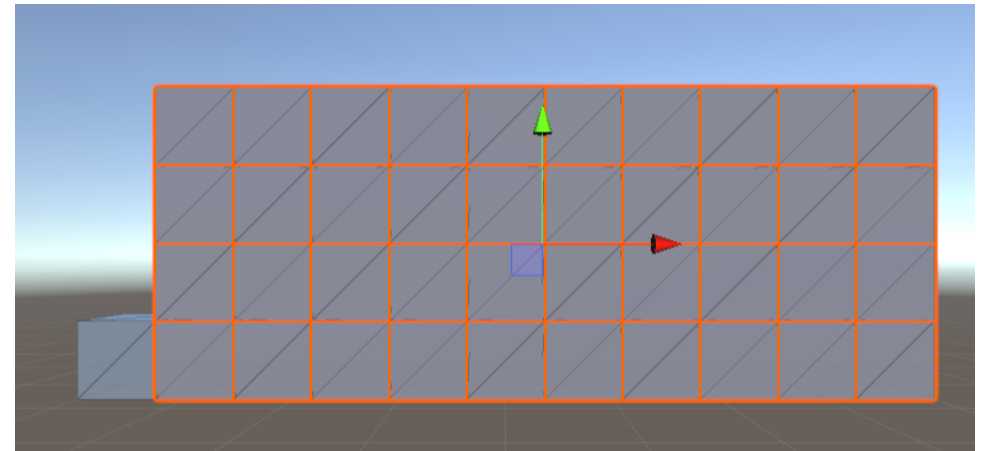
1. `public static Object Instantiate(Object original);`
2. `public static Object Instantiate(Object original, Transform parent);`
3. `public static Object Instantiate(Object original, Transform parent, bool instantiateInWorldSpace);`
4. `public static Object Instantiate(Object original, Vector3 position, Quaternion rotation);`
5. `public static Object Instantiate(Object original, Vector3 position, Quaternion rotation, Transform parent);`
6. `Destroy(GameObject go);`

Create objects

```
using UnityEngine;

public class Wall : MonoBehaviour
{
    public GameObject block;
    public int width = 10;
    public int height = 4;

    void Start()
    {
        for (int y = 0; y < height; ++y)
        {
            for (int x = 0; x < width; ++x)
            {
                Instantiate(block, new Vector3(x+1, y, 0), Quaternion.identity);
            }
        }
    }
}
```



Input.GetKeyDown

- Part of **Input class**, used to detect **keyboard input**
- GetKeyDown() method:

```
public static bool GetKeyDown(key);
```

- Returns true **only during the frame** when the user **presses down** the specified key;

Input.GetKeyDown

- Usage Examples:
- public static bool **GetKeyDown**(string name);

```
if (Input.GetKeyDown("space")){ //do something }
```

- public static bool **GetKeyDown**(KeyCode key);

```
if (Input.GetKeyDown(KeyCode.Space)) { //do something }
```

- **KeyCode** values map **directly to physical keys** on the keyboard and are **case-sensitive and platform-independent**.

Instantiate and Destroy Example

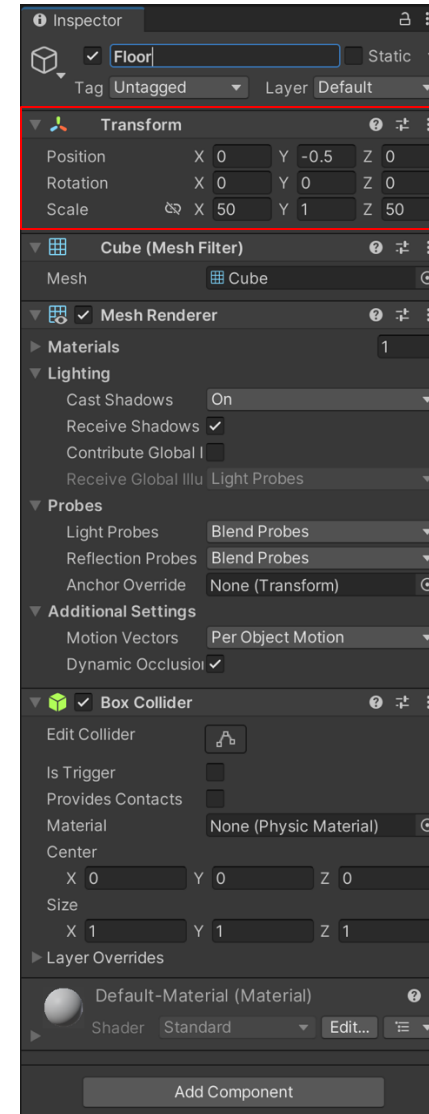
```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class infovizScript_1 : MonoBehaviour {
    public GameObject spawn;
    void Update()
    {
        if (Input.GetKeyDown(KeyCode.C))
        {
            GameObject go = Instantiate(
                spawn, new Vector3(2, 0.5F, 0),
                new Quaternion());
        }
    }
}
```

```
        go.tag = "Respawn";
    }
    if (Input.GetKeyDown(KeyCode.D)){
        GameObject[] res =
        GameObject.FindGameObjectsWithTag("Respawn");
        foreach(GameObject g in res)
            Destroy(g);
    }
}
```

Transform class

- The **Transform class** is used to store and manipulate the **position**, **rotation**, and **scale** of a GameObject.
- Every object in a Unity scene has a Transform component by default.
- An important **property**:
 - `transform.position`
- Two important **methods**:
 - `transform.Translate()`
 - `transform.Rotate()`



Transform.position

- The **transform.position** property defines the **world-space position** of a GameObject in the scene.
- Stores and returns the object **position in global (world) coordinates**.
- Can be **read** to get the current position, or **assigned** to move the object to a new location.

```
// Get the current position of an Object  
Vector3 currentPos = transform.position;
```

```
// Move the object to a new position  
Vector3 pos = new Vector3(0, 5, 10);  
transform.position = pos;
```

```
// Move it up by 1 unit  
transform.position += Vector3.up;
```

Transformations



- **Translation – Moving Object** (moves the object by the given amount):

```
1. public void Translate(Vector3 translation, Space relativeTo = Space.Self);
```

```
2. public void Translate(float x, float y, float z, Space relativeTo = Space.Self);
```

- **Rotation - Rotate Object** (rotates the object around specified axes):

```
1. public void Rotate(Vector3 eulerAngles, Space relativeTo = Space.Self);
```

```
2. public void Rotate(float xAngle, float yAngle, float zAngle, Space relativeTo = Space.Self);
```

```
3. public void Rotate(Vector3 axis, float angle, Space relativeTo = Space.Self);
```

- The “Space relativeTo” parameter defines the frame of reference for the transformation (which coordinate system will be used to apply the movement or rotation):
 - **Space.Self**: transformation applied using the **object’s own axes**.
 - **Space.World**: transformation applied using **global/world axes**.



Suggestion: use Space.Self for local controls (e.g., player movement), and Space.World when working with global positioning or environmental effects.



Transformation Example (1):

- A script that controls the position of a game object in the scene by using the key LeftArrow and RightArrow.

```
using UnityEngine;
using System.Collections;

public class Movimenti : MonoBehaviour {
    public int moveSpeed;

    // Use this for initialization

    void Start () {}
    // Update is called once per frame
    void Update () {
        if (Input.GetKey(KeyCode.LeftArrow)) {
            transform.Translate( Vector3.forward * moveSpeed);}//endif
        if (Input.GetKey (KeyCode.RightArrow)) {
            transform.Translate(-Vector3.forward * moveSpeed);}//endif
        }//endupdate
    }//end class
```

Time control

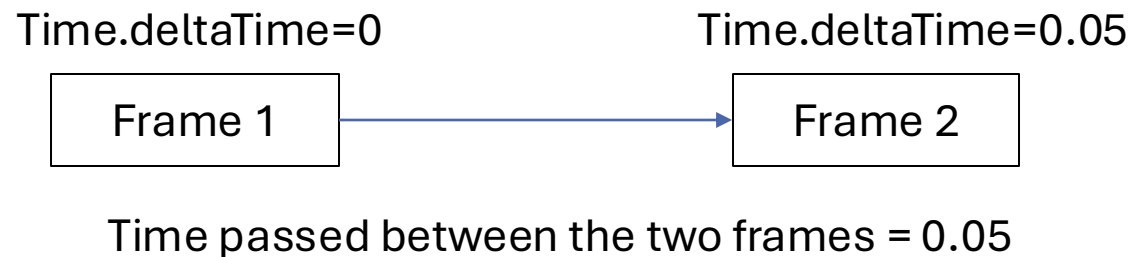
```
transform.Translate(new Vector3(0,0,movespeed));
```

- This line moves the object along the **Z axis** by moveSpeed units on **every frame update** (Update()).
- The effect of translating the object is related to the FPS (frame per second).
- The **faster the frame rate (FPS)**, the **more often** this movement occurs per second.
- What is the side effect of such a condition? Consider movespeed=5

Frame Rate–Independent Movement: Time.deltaTime()

```
transform.Translate(Vector3.forward * moveSpeed * Time.deltaTime);
```

- *Time* class provides an interface to get time information from Unity.
- Time.deltaTime represents the **time (in seconds)** that passed **since the last frame** *Read-only*.
- This ensures that movement is **smooth, consistent**, and **frame rate independent**.
- With Time.deltaTime, the object is moved by **moveSpeed units per second**, not per frame.



Transformation Example (2):

```
...  
// Update is called once per frame  
void Update () {  
  
    if (Input.GetKey(KeyCode.LeftArrow)) {  
        transform.Translate(Vector3.forward * moveSpeed * Time.deltaTime );  
    }  
    if (Input.GetKey (KeyCode.RightArrow)) {  
        transform.Translate(-Vector3.forward * moveSpeed * Time.deltaTime);  
    }  
}  
}
```

Now run the script again!!

Position VS Translate()



- What is the difference between the two function calls 1 and 2 below?

```
Vector3 newpos = new Vector3(Random.value, Random.value, Random.value);  
  
// Calls 1  
transform.position = newpos;  
  
// Calls 2  
transform.Translate(newpos);
```



Position VS Translate()



	transform.position	transform.Translate()
Movement type	Absolute sets exact world position	Relative moves object by a given amount
Overwrites previous position?	Yes	No – adds to the current position
Uses world or local space?	World space only	Local by default, can specify Space.World
Usage	Teleporting, initializing, reset	Smooth movement, animation, player control

```
Vector3 newpos = new Vector3(Random.value, Random.value, Random.value);  
  
// 1. Absolute positioning  
transform.position = newpos;  
  
// 2. Relative translation  
transform.Translate(newpos);
```

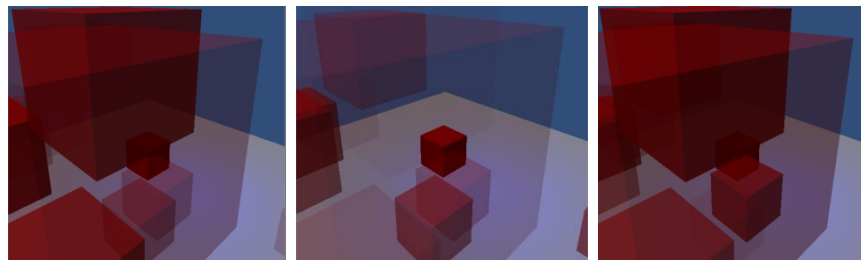
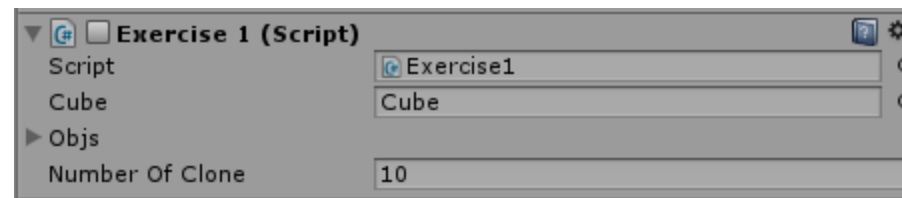
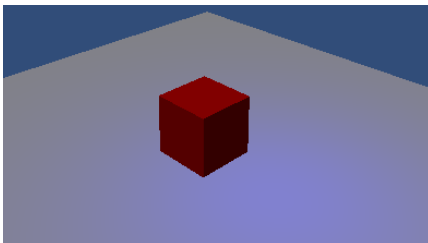


Esercizio 1

1. Instanziare delle primitive nella scena a proprio piacere.
2. Assegnare uno script Movements.cs alla camera
3. Instanziare una variabile ***pubblica movespeed*** e una ***turnspeed***.
4. Fare in modo che alla pressione dei tasti freccia (*`Input.GetKey(KeyCode.UpArrow)`*) della tastiera corrispondano i seguenti movimenti della camera:
 - ↑ - muove la camera in avanti (`Vector3.forward`)
 - ↓ - muove la camera indietro (`-Vector3.forward`)
 - → - ruota la camera a destra (`Vector3.up`)
 - ← - ruota la camera a sinistra (`-Vector3.up`)

Esercizio 2

1. Creare un cubo nella scena e aggiungere un materiale di colore rosso e shader *Transparent*.
2. Scrivere uno script che prima di essere abilitato permetta di selezionare un numero intero *numOfClone* e selezionare un cubo presente nella scena.
3. Una volta abilitato (**Start**), istanziare in posizioni random i cloni dell'oggetto stabilito e ponga in un array
 - **Random.value**: genera un valore random $\in [0,1]$
 - Arrays
 - Dichiarazione: `public GameObject[] clones`
 - Inizializzazione: `clones = new GameObject[numOfClone]`
4. Ad ogni frame (**Update**) aggiornare il valore alpha del materiale associato a ciascun oggetto con un valore random.
 1. Accedere al materiale: `color = GetComponent<Renderer>().material.color`
 2. Accedere al canale alpha: `color.a`



Transform.LookAt()

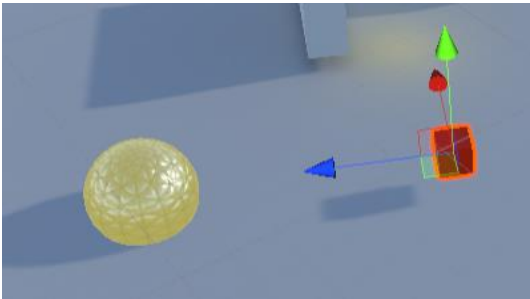
- The LookAt() function rotates the transform so the forward vector points at target's/current position:

```
1. public void LookAt(Transform target, Vector3 worldUp = Vector3.up);  
2. public void LookAt(Vector3 worldPosition, Vector3 worldUp = Vector3.up);
```

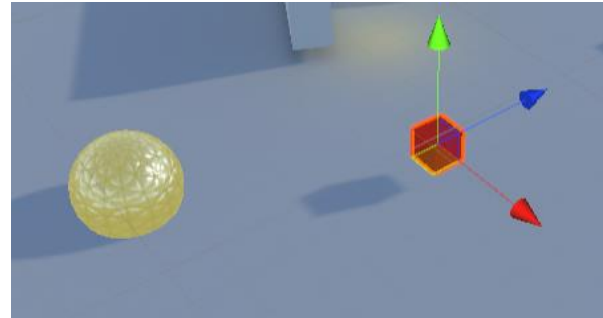
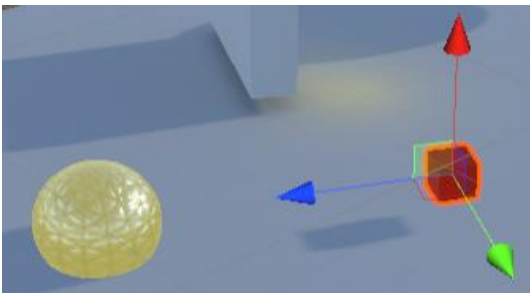
- Rotates the object so that its **local Z axis** points directly at the specified **target**.
- The optional worldUp parameter defines what direction should be considered "**up**" for the object.
- By default, is Vector3.up (the **world Y axis**).

Transform.LookAt(): worldUp

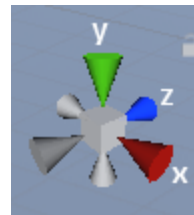
Vector3.up



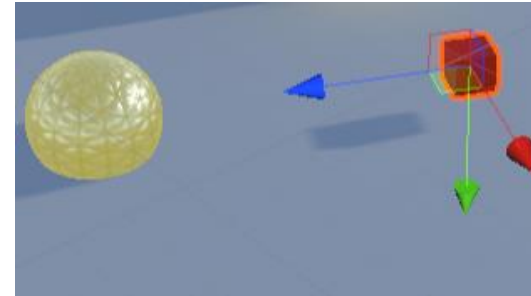
Vector3.right



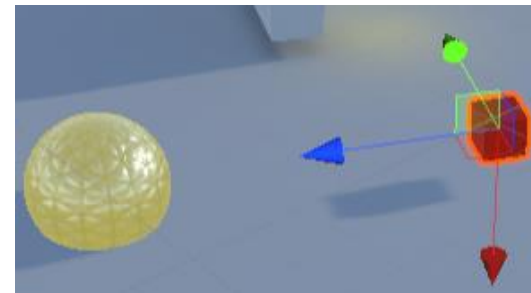
Transform.LookAt
(mira.transform, **Vector3.X**);



Vector3.down



Vector3.left



Coroutine

- Normally, when you call a function in Unity, it runs **from start to finish in a single frame**.
- This means that any action taking place in a function must happen within a single frame update;
- A **coroutine** is a special kind of function that can **pause execution (returning the control to Unity)** at a certain point and **resume** on the next frame (or after a delay):
 - Coroutines are declared with a return type of IEnumerator:

```
IEnumerator MyCoroutine() {  
    // code before pause  
    yield return null;}
```

- To start a coroutine:

```
StartCoroutine(MyCoroutine());
```

- The **yield return** is the point at which execution will pause and be resumed the following frame.

Coroutine example (1)

```
Renderer rend;  
void Start()  
{  
    rend = GetComponent<Renderer>();  
}  
void Update () {  
    if (Input.GetKeyDown ("f")) {  
        simple_Fade();}  
    if (Input.GetKeyDown ("g")) {  
        StartCoroutine(coroutine_Fade())  
    }  
    // Simple method  
void simple_Fade(){  
    for (float f = 1f; f >= 0; f -= 0.01f) {
```

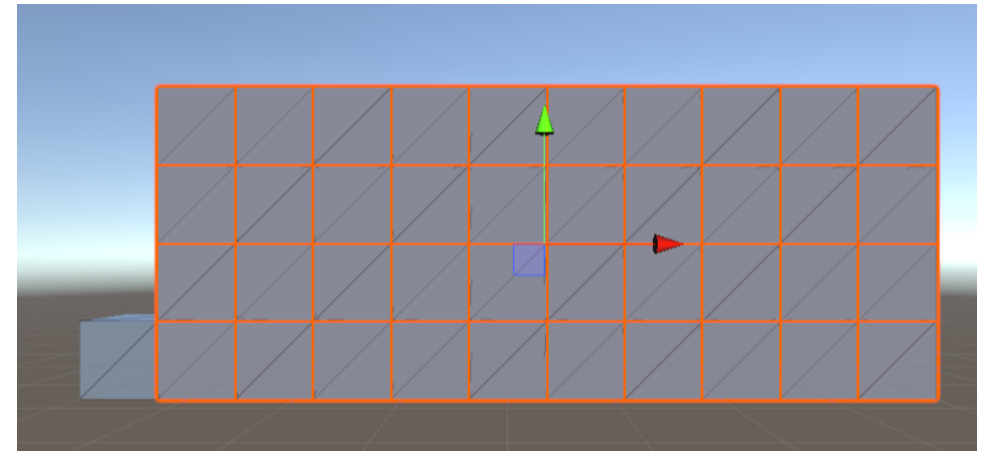
```
        Color c = rend.material.color;  
        c.a = f;  
        rend.material.color = c; }  
}  
//Coroutine  
IEnumerator coroutine_Fade() {  
    for (float f = 1f; f >= 0; f -= 0.01f) {  
        Color c = rend.material.color;  
        c.a = f;  
        rend.material.color = c;  
        yield return null;  
    }  
}
```

Coroutine example (2)

```
public class Wall : MonoBehaviour
{
    public GameObject block;
    public int width = 10;
    public int height = 4;
    public float delay = 0.1f; // ritardo tra un mattone e l'altro

    void Start() {
        StartCoroutine(BuildWall());
    }

    IEnumerator BuildWall(){
        for (int y = 0; y < height; ++y){
            for (int x = 0; x < width; ++x){
                Instantiate(block, new Vector3(x + 1, y, 0), Quaternion.identity);
                yield return new WaitForSeconds(delay);
            }
        }
    }
}
```



OnMouse Events

- Unity supports **event-based programming**, and some of the most common built-in events are related to **mouse interactions** with objects in the scene.
- Particularly useful the classic events triggered by the mouse called when the mouse enters the [GUIElement](#) or [Collider](#) :
- Some Mouse Events:
 - OnMouseEnter: Sent when the mouse enters an element or one of its descendants.
 - OnMouseExit: Sent when the mouse is not any longer over an element
 - OnMouseOver: Sent when the mouse enters an element.
 - OnMouseDown: Sent when the user presses a mouse button.

OnMouseEvents example

```
void OnMouseOver( ){
```

```
    Color color = GetComponent<Renderer>().material.color;  
    color.a = 1F;  
    GetComponent<Renderer>().material.color = color;
```

```
}
```

```
void OnMouseExit( ){
```

```
    Color color = GetComponent<Renderer>().material.color;  
    color.a = 0.5F;  
    GetComponent<Renderer>().material.color = color;
```

```
}
```

```
void OnMouseDown( ){
```

```
    Color color = GetComponent<Renderer>().material.color;  
    color.r = Random.value;  
    color.g = Random.value;  
    color.b = Random.value;  
    color.a = 1f;  
    GetComponent<Renderer>().material.color = color;
```

```
}
```

Physic in



Rigidbody and Colliders

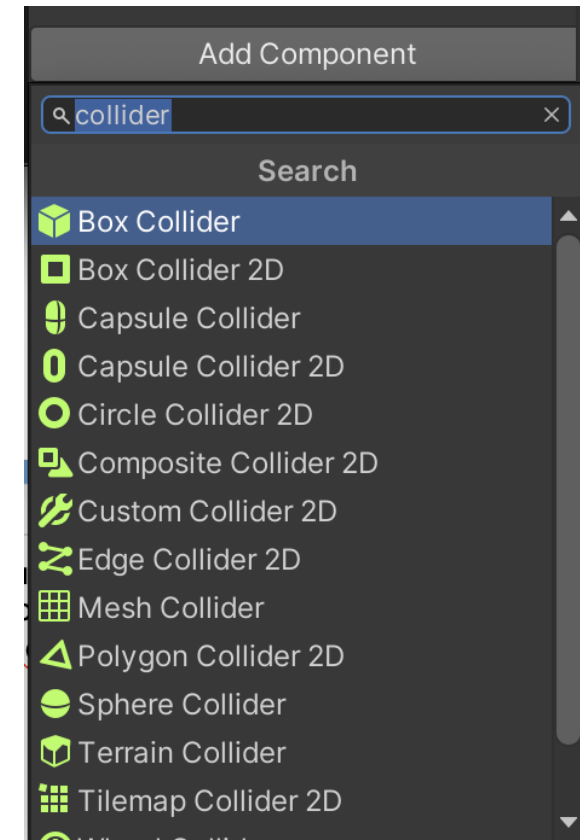
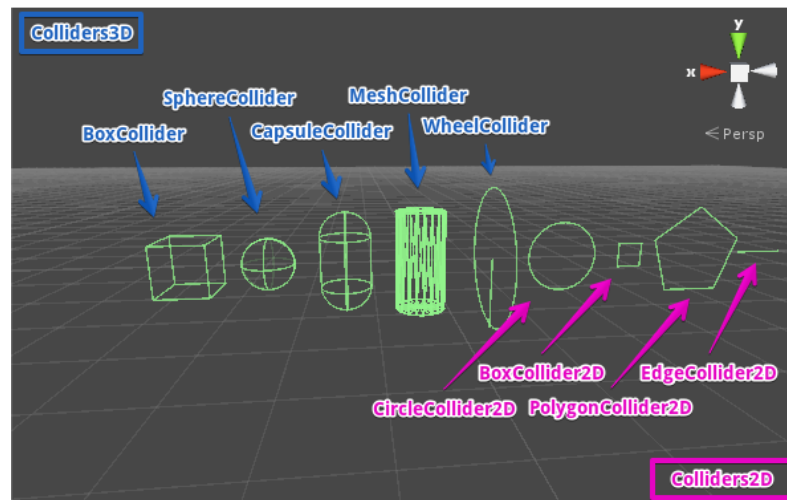
- A **Rigidbody** component enables **physical behaviour** for a GameObject.
 - With a Rigidbody attached, the object will immediately respond to gravity and interacts with other objects via **collisions** and **forces**.
 - When using Rigidbody avoid modifying transform.position directly. Instead, use physics methods like AddForce() or velocity to move the object.
This allows Unity's **physics engine** to manage realistic movement.
- A **Collider** components define the **physical shape** of the object for collision detection.
- It is **invisible** in the game but used by Unity to calculate collisions, triggers and contacts.
- The collider **doesn't have to match** the visual mesh perfectly. A **simplified shape** (box, sphere, capsule) is often **more efficient** and sufficient for gameplay.

Few words on Colliders

- **Colliders** control **how accurately** collisions are detected between GameObjects.
- More **precise collision detection** can improve realism but comes at a **higher performance cost**.
- In real-time applications, you often need to find the right **balance between accuracy and efficiency**.
- Solutions are different:
 - **Primitive Collider**: primitives are used to approximate the shape of the object
 - **Mesh Collider**: Uses the actual **3D mesh** of the object for collisions. More **accurate**, but **computationally expensive**.
 - **Compound collider**: Combines multiple **primitive colliders** to approximate a more complex shape. Good **compromise** between **performance and realism**.

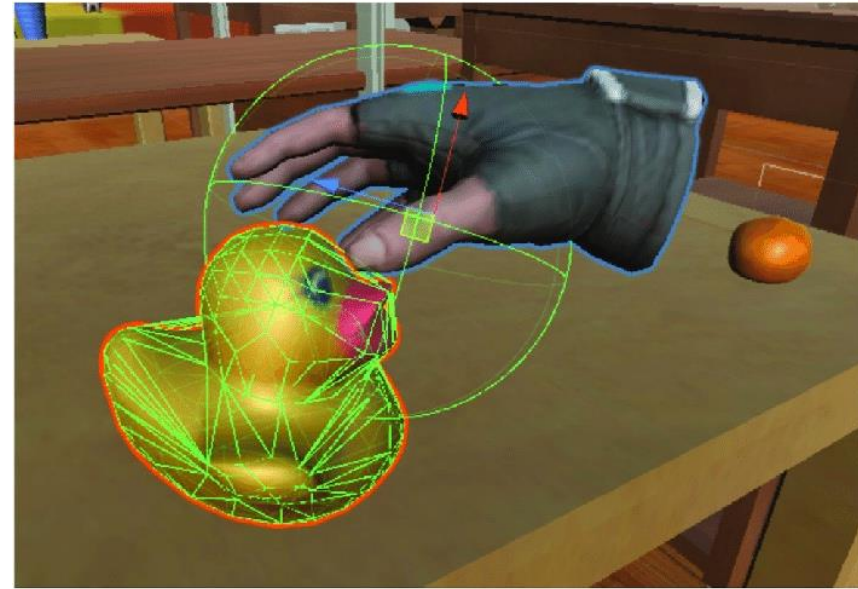
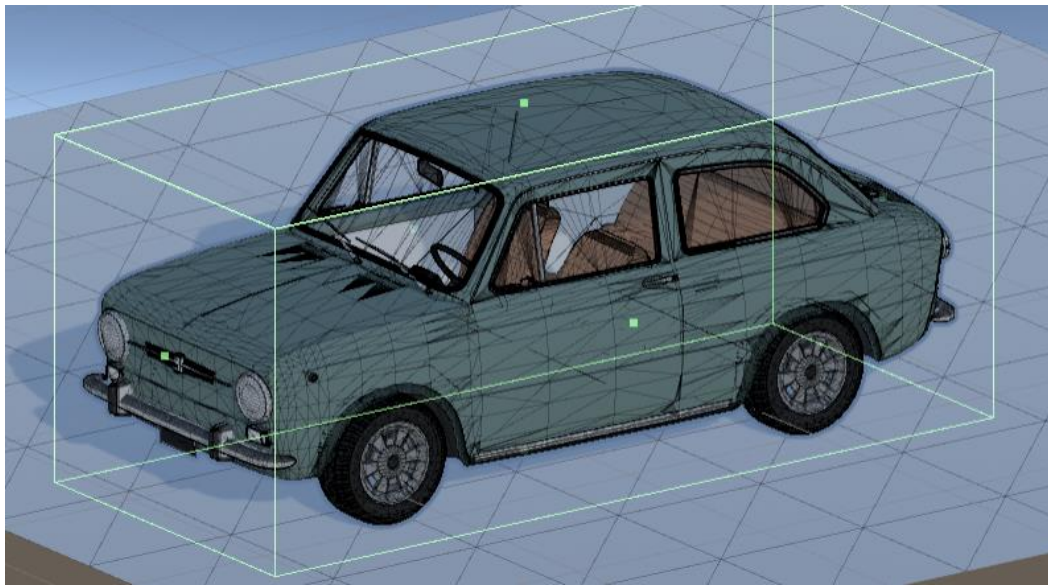
Primitive Colliders

- Unity offers a set of built-in primitive colliders — such as Box, Sphere, Capsule, and Cylinder — to efficiently approximate the shape of most simple objects.



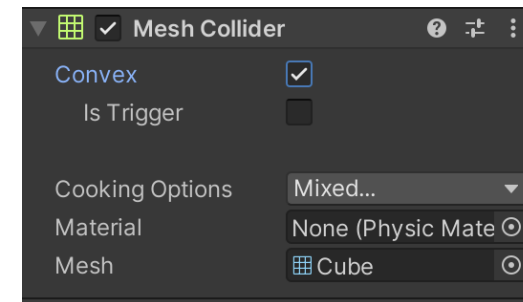
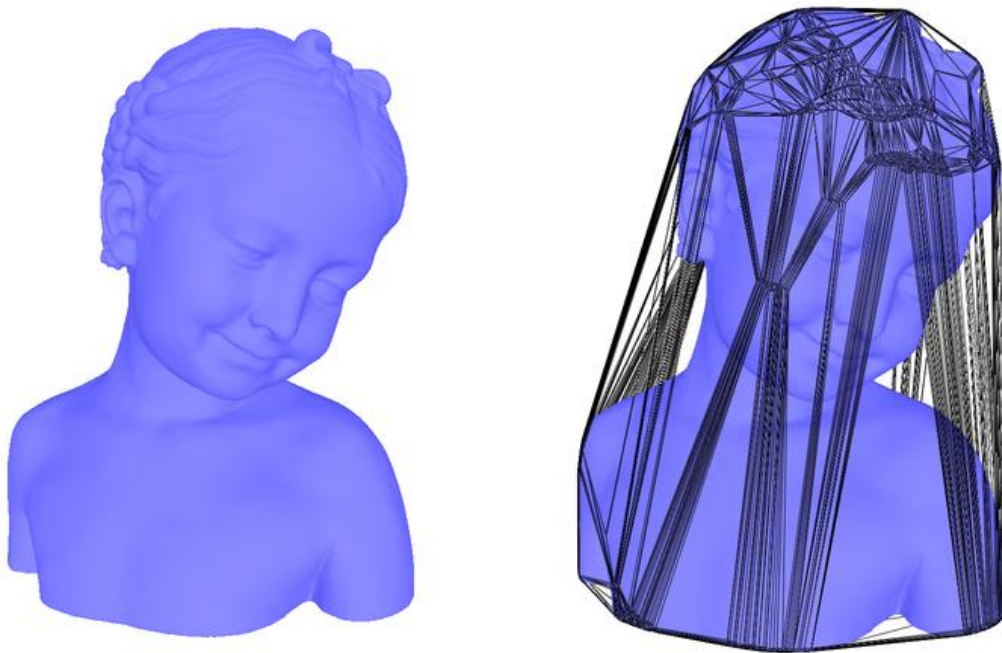
Mesh Collider

- Collision detection is based on the object's full mesh: accurate, but computationally expensive!



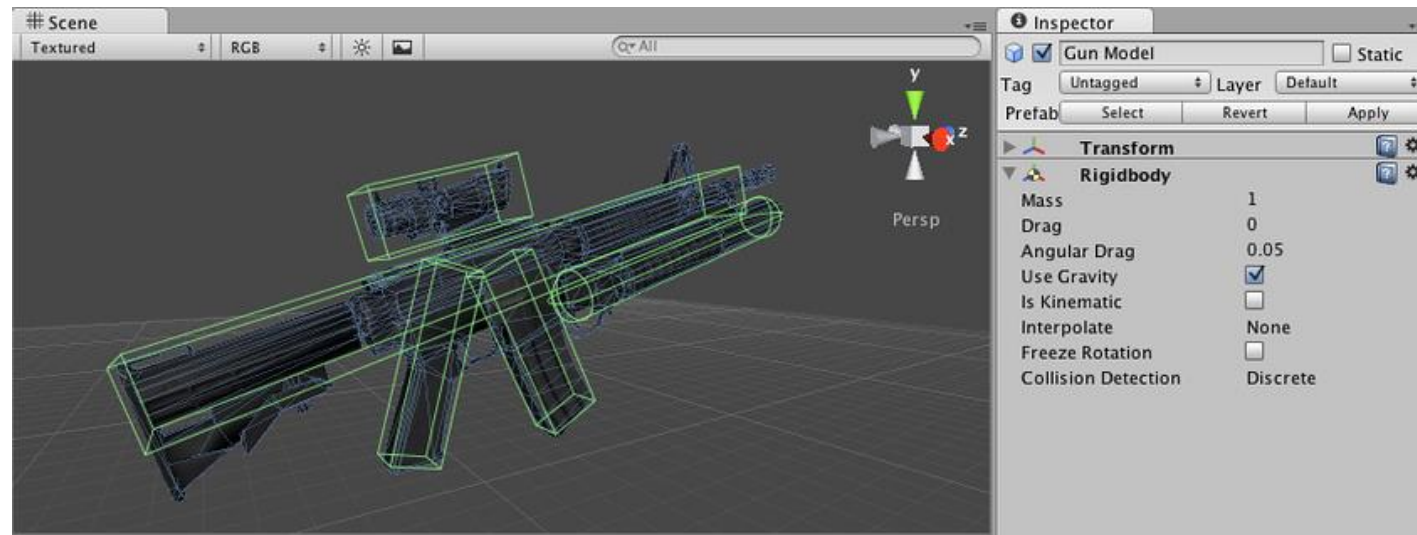
Convex Hull

- A **Convex Hull** is the **smallest convex shape** that completely encloses all the vertices of a mesh.
 - Can be enabled by checking the “**Convex**” option in the **Mesh Collider component** in the Inspector.



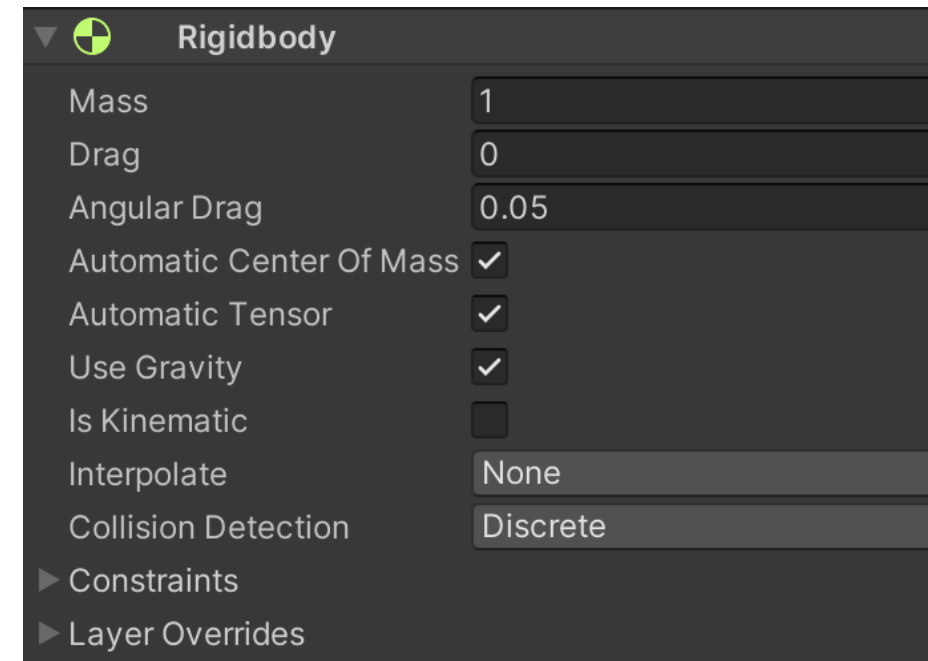
Compound Colliders

- A **compound collider** is created by combining multiple **primitive colliders** (e.g., box, sphere, capsule).
- Together, these colliders **approximate the shape** of a complex 3D model **without the performance cost** of a mesh collider.
- Ideal for dynamic objects that require **both efficiency and reasonable physical accuracy**.



Scripting and Physics

- When working with **physics-enabled objects**, you should **avoid moving them directly** using translate/rotation/position operations.
- Instead, objects with a **Rigidbody** component should be moved using the **physics system** (e.g., `AddForce()`, `velocity`, or `MovePosition()`), so that Unity's physics engine can handle collisions and interactions realistically.
- Actually, when an object has a Rigidbody component, has its own:
 - Mass
 - Drag
 - Angular Drag
 - Gravity(and other properties from the inspector)



Scripting and Physics

- In some situations, we may want to **bypass physics simulation** and control an object's movement **directly via scripting** (e.g., using `Translate()` or `Rotate()`), while still remaining consistent with the scene.
 - A common example is a **sliding door** or an **elevator**, where applying realistic physical forces would be unnecessarily complex.
- Unity provides the “**Is Kinematic**” option on the Rigidbody component.
 - The object **does not respond** to physics forces or collisions.
 - You can safely **move it using Transform** methods without interfering with the physics engine.
 - The object will **not be affected by gravity**.

Collisions and Events

- When a GameObject has **both a Rigidbody and a Collider**, Unity can automatically trigger **collision events** through scripting.
- These events allow you to **respond to physical interactions** between objects in the scene.
- Unity provides the following collision event methods:
 - ***Collider.OnCollisionEnter***: called once when the collision begins
 - ***Collider.OnCollisionStay***: called every frame while the collision is ongoing
 - ***Collider.OnCollisionExit***: called once when collision ends

- How to:

```
Void OnCollisionEnter(Collision collision){  
    //do something in case of collisions  
}
```

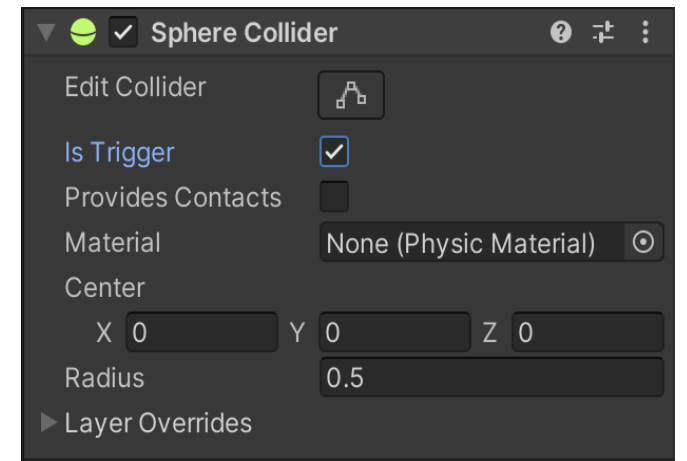
Collisions and Events

```
Void OnCollisionEnter(Collision collision)
```

- When a collision occurs, Unity automatically calls the OnCollisionEnter() method and passes a **Collision object** as a parameter.
- This Collision object provides detailed **information about the collision** that just happened.
- The key properties of the Collision class are:
 1. **collider** The Collider we hit (Read Only).
 2. **contacts** The contact points generated by the physics engine.
 3. **gameObject** The GameObject associated with the other collider (read-only)
 4. **impulse** The total **impulse force** applied to resolve the collision.
 5. **relativeVelocity** The relative linear velocity between the two colliding objects (read-only).
 6. **rigidbody** The Rigidbody attached to the other object, or null if it doesn't have one.
 7. **transform** The Transform of the object we hit (read only).

Triggers

- You can also use the physics engine **simply to detect** when one collider enters the space of another **without creating a collision**.
- A **Trigger** is a **Collider** component with the **"Is Trigger"** property enabled.
- Unlike regular colliders, it does **not physically block** other objects — instead, it allows them to **pass through**, and **detects their presence**.
- When another collider enters the trigger's space, Unity automatically calls one of the following **event methods**::
 - void OnTriggerEnter(Collider other)
 - void OnTriggerStay(Collider other)
 - void OnTriggerExit(Collider other)
- Each method receives the **other Collider** involved in the interaction.



Triggers and Collision messages

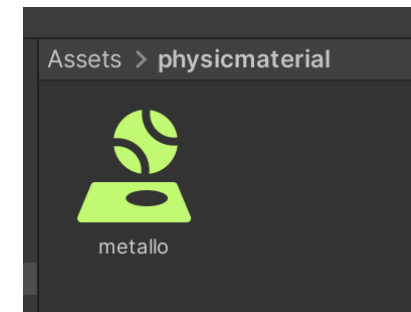
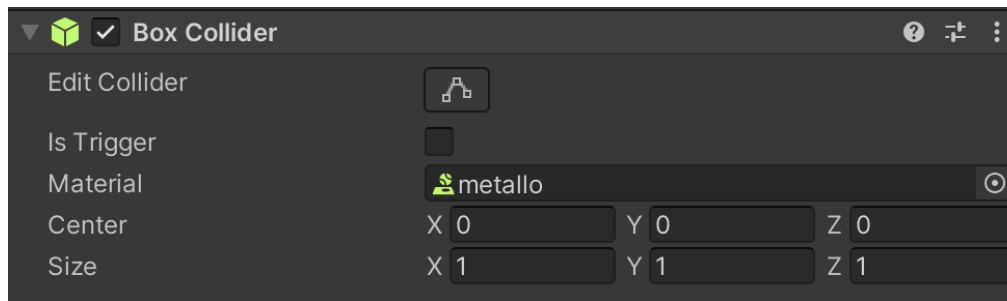
Collision detection occurs and messages are sent upon collision						
	Static Collider	Rigidbody Collider	Kinematic Rigidbody Collider	Static Trigger Collider	Rigidbody Trigger Collider	Kinematic Rigidbody Trigger Collider
Static Collider		Y				
Rigidbody Collider	Y	Y	Y			
Kinematic Rigidbody Collider		Y				
Static Trigger Collider						
Rigidbody Trigger Collider						
Kinematic Rigidbody Trigger Collider						

Trigger messages are sent upon collision						
	Static Collider	Rigidbody Collider	Kinematic Rigidbody Collider	Static Trigger Collider	Rigidbody Trigger Collider	Kinematic Rigidbody Trigger Collider
Static Collider					Y	Y
Rigidbody Collider				Y	Y	Y
Kinematic Rigidbody Collider				Y	Y	Y
Static Trigger Collider		Y	Y		Y	Y
Rigidbody Trigger Collider	Y	Y	Y	Y	Y	Y
Kinematic Rigidbody Trigger Collider	Y	Y	Y	Y	Y	Y

Colliders on gameobjects with no rigidbody are considered static.

Physic Material

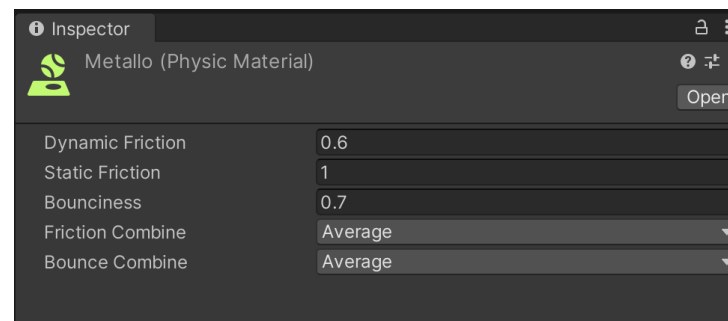
- A **Physic Material** allows you to control how objects **behave on contact**, specifically by adjusting friction (sliding resistance) and bounciness (rebound effect).
- To create a Physic Material **Assets > Create > Physic Material** from the menu bar.
- Apply it by dragging the Physic Material onto a **Collider component** in the scene.



Physic Material

- The **Physic Material** is used to regulate the effects of friction and bounce when two objects collide.

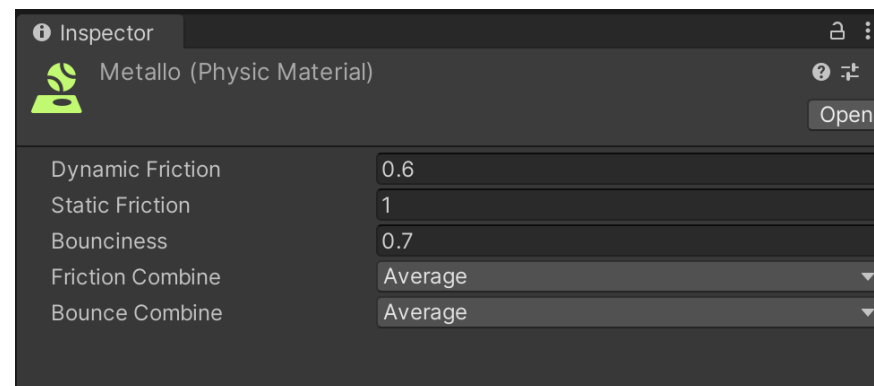
Dynamic Friction	The friction used when already moving. Usually a value from 0 to 1. A value of zero feels like ice, a value of 1 will make it come to rest very quickly unless a lot of force or gravity pushes the object.
Static Friction	The friction used when an object is laying still on a surface. Usually a value from 0 to 1. A value of zero feels like ice, a value of 1 will make it very hard to get the object moving.
Bounciness	How bouncy is the surface? A value of 0 will not bounce. A value of 1 will bounce without any loss of energy, certain approximations are to be expected though that might add small amounts of energy to the simulation.



Physic Material

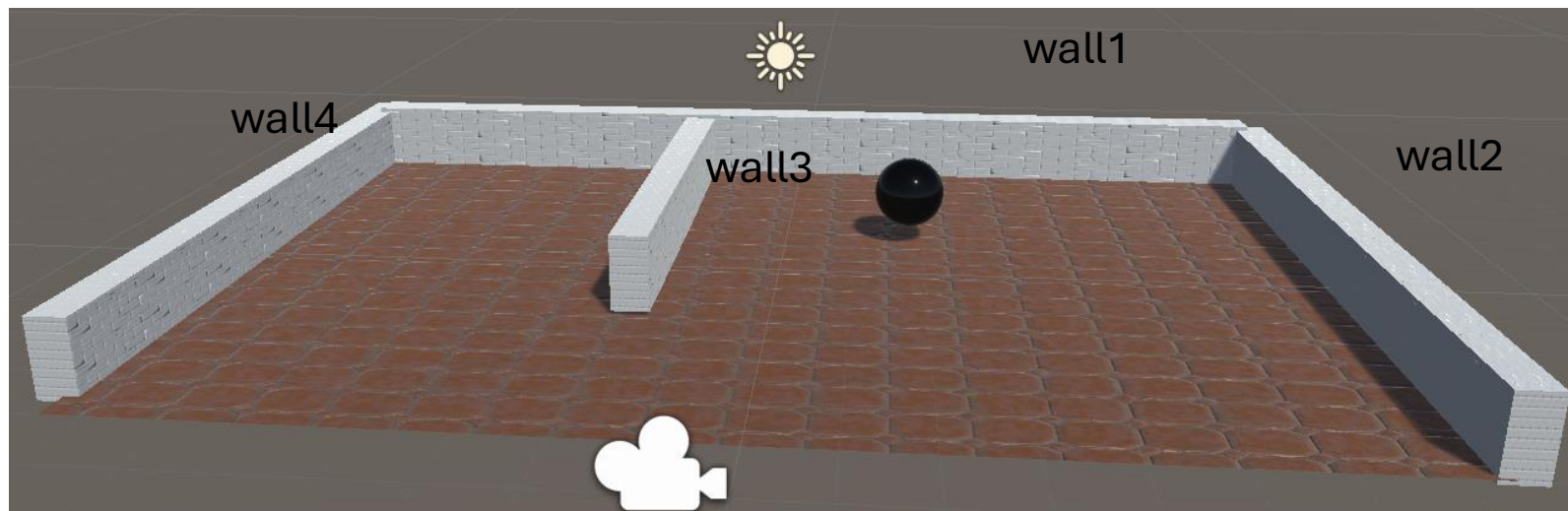
- The **Physic Material** is used to regulate the effects of friction and bounce when two objects collide.

Friction Combine	How the friction of two colliding objects is combined.
- Average	The two friction values are averaged.
- Minimum	The smallest of the two values is used.
- Maximum	The largest of the two values is used.
- Multiply	The friction values are multiplied with each other.
Bounce Combine	How the bounciness of two colliding objects is combined. It has the same modes as Friction Combine Mode



Esercizio 3 - Riepilogo

- Creare una biglia nera che si muove all'intorno di una stanza.
- I movimenti devono essere gestiti utilizzando i tasti freccia.
- Quando collido con l'oggetto wall 1 la biglia cambia colore.
- Al termine della collisione con l'oggetto wall 4, l'oggetto wall 4 scompare.
- Quando la biglia resta in collisione con l'oggetto wall 2, l'oggetto wall 2 cambia colore utilizzando la funzione Lerp.
- Rendere l'oggetto wall 3 'is triggered' e cambiare il colore di wall1 quando collido.
- Muovere la camera insieme alla biglia sfruttando la funzione LookAt() oppure sfruttando la posizione dell'oggetto in movimento.



Graphic User Interface GUI

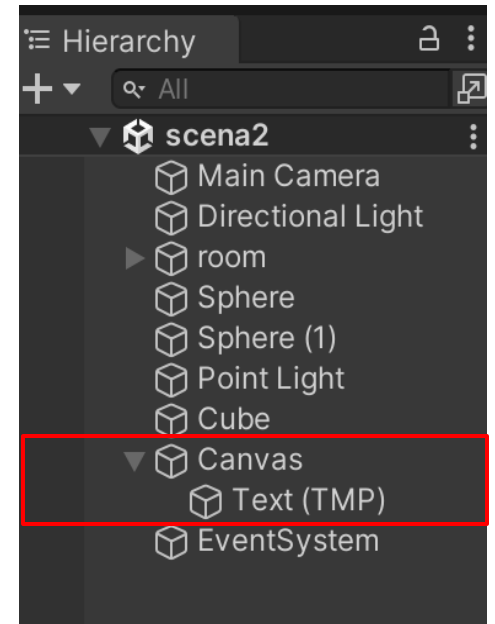


User Interface

- **Unity UI** is a **GameObject-based toolkit** for building user interfaces in games and applications.
- It is a GameObject-based UI system that uses Components and the Game View to arrange, position, and style user interfaces.
- It uses familiar concepts like **Components**, **Hierarchy**, and the **Game View** to create and arrange UI elements such as buttons, text, panels, and sliders.
- Unity UI is designed for runtime interfaces — *you cannot use it to modify or extend the Unity Editor UI itself.*
- UI elements are organized under a **Canvas** and rendered as part of the scene.

Canvas

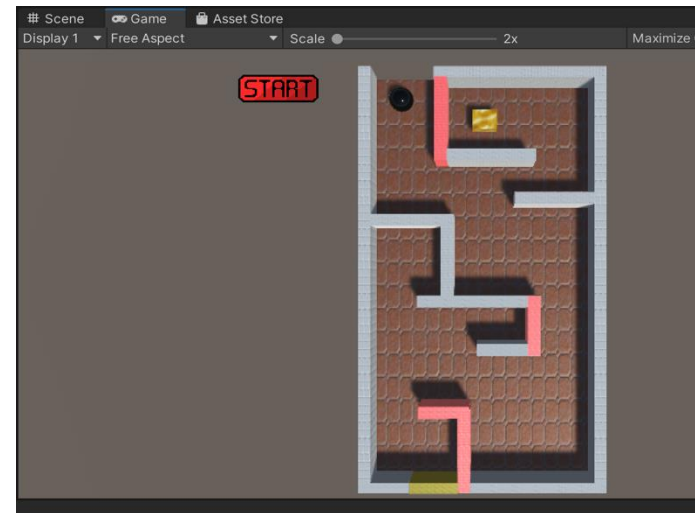
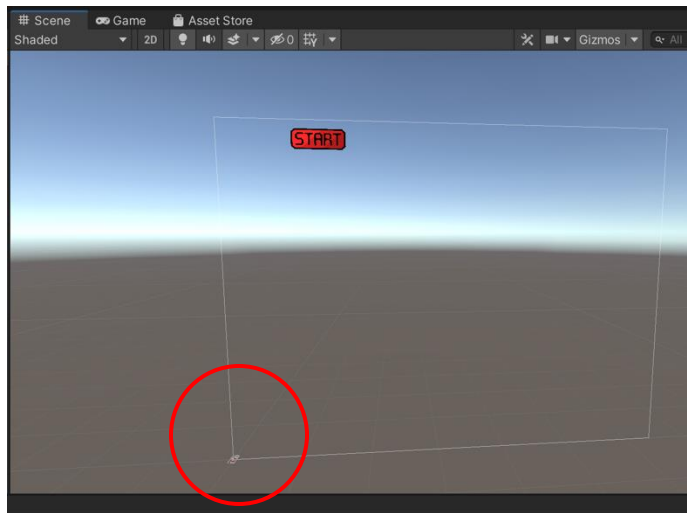
- The **Canvas** is the root element for all UI elements in Unity: it defines the **space where UI elements are drawn** and manages how they are rendered.
- The Canvas itself is a **GameObject** with a **Canvas component**, and **all UI elements must be its children** in the hierarchy.
- To create a exist and go to **GameObject** → **UI** → **Canvas**
- When you create any UI element (e.g., Button, Text), Unity **automatically creates a Canvas** if one doesn't exist and places the UI element **as a child of the Canvas**.



UI elements will not render in the Game view unless they are under a Canvas!!

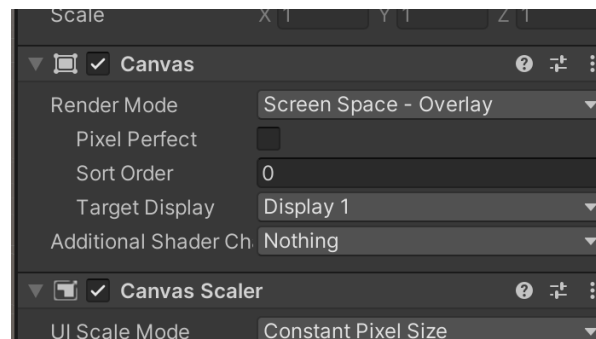
Canvas

- The **Canvas** appears as a **visible rectangle** in the **Scene View**, representing the UI layout area.
- This allows you to **position and edit UI elements** directly in the Scene View without needing to constantly switch to or rely on the **Game View**.
- UI elements inside the Canvas can be resized, aligned, and arranged just like 3D objects, but using the **2D layout system**.



Canvas Render Modes

- The **Canvas** component offers different **Render Modes**, which determine **how and where** UI elements are drawn in relation to the scene. Three types of Render Mode:
- **Screen Space – Overlay**: This render mode places UI elements on the screen rendered on top of the scene. If the screen is resized or changes resolution, the Canvas will automatically change size to match.
- **Screen Space – Camera**: in this render mode the Canvas is placed a given distance in front of a specified **Camera**. The Camera settings affect the appearance of the UI, and the amount of perspective distortion can be controlled by the Camera **Field of View**.
- **World Space**: the Canvas will behave as any other object in the scene. UI elements appear **within the 3D environment** and can be occluded or interact with other 3D objects. This mode is for **diegetic UI** (e.g., screens in the game world, signs, labels).



Basic Layout

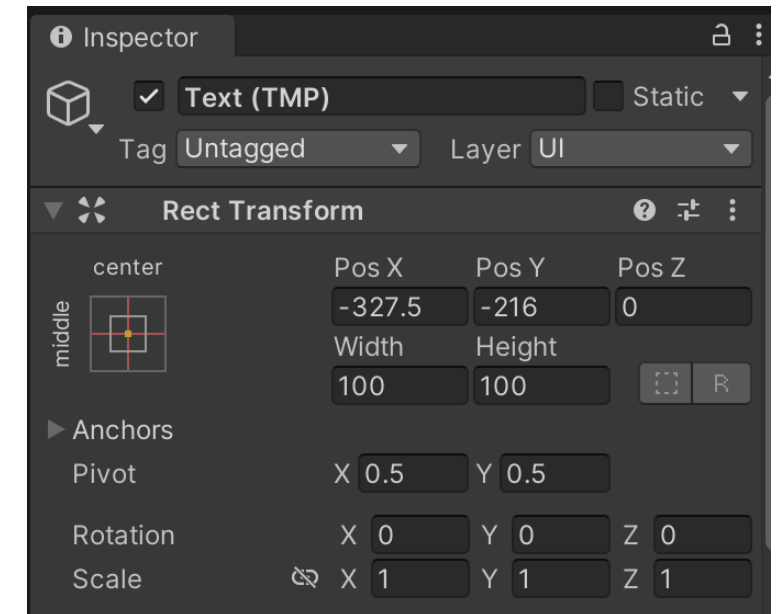
- All UI elements are treated as **rectangles** for layout and positioning within the Canvas.
- You can create a UI image via **GameObject** → **UI** → **Image**.
- The **Rect Tool** is used to **move, resize, and rotate** UI elements directly in the **Scene View**.
- **Rect Tool** provides handles for adjusting the element's dimensions and orientation.
- The Rect Tool respects the current **Pivot Mode** and **Transform Space**, which can be set in the Unity toolbar.
- For most UI tasks, it's best to work with the settings **Pivot** and **Local** enabled for precise and predictable manipulation.
- **Pivot** → manipulates UI relative to the anchor/pivot point.
- **Local** → uses the element's local coordinate system.

Rect Tool



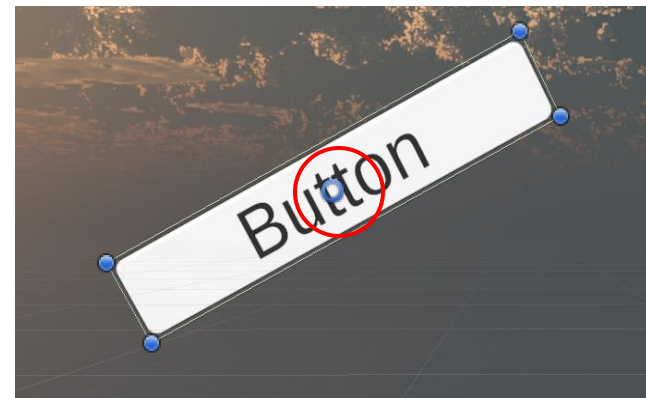
Rect Transform

- **Rect Transform** is a specialized version of the standard Transform component, used **exclusively for UI elements**.
- Like Transform, it still provides **position**, **rotation** and **scale**.
- In addition, it introduces:
 - **Width** and **height** to define the **dimensions of the UI rectangle**.
 - **Anchors**, **pivot**, and **offsets**, which essential for flexible and responsive UI layout
- RectTransform gives precise control over **UI layout and scaling**, especially when adapting interfaces to different screen sizes and resolutions.



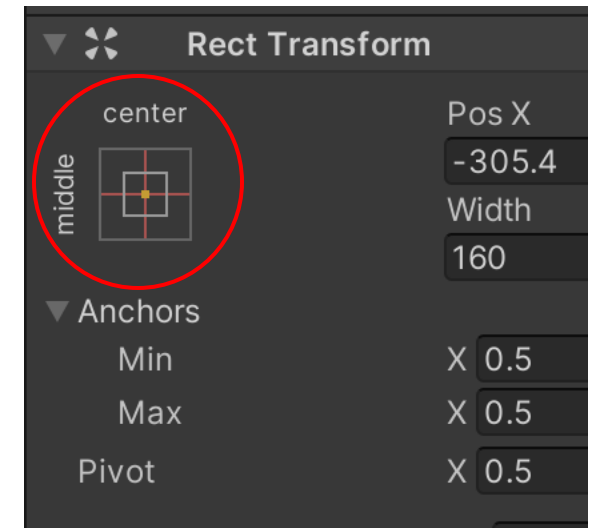
Pivot in Rect Transform

- When the toolbar Pivot button is set to **Pivot** mode, the pivot of a Rect Transform can be moved in the Scene View.
- When the **toolbar is set to Pivot mode**, you can **visually move the pivot point** in the **Scene View** using the **Rect Tool**.
- The pivot is represented as **normalized coordinates** (0 to 1), relative to the element of the rectangle:
 - (0, 0) → bottom-left corner
 - (0.5, 0.5) → center (default)
 - (1, 1) → top-right corner

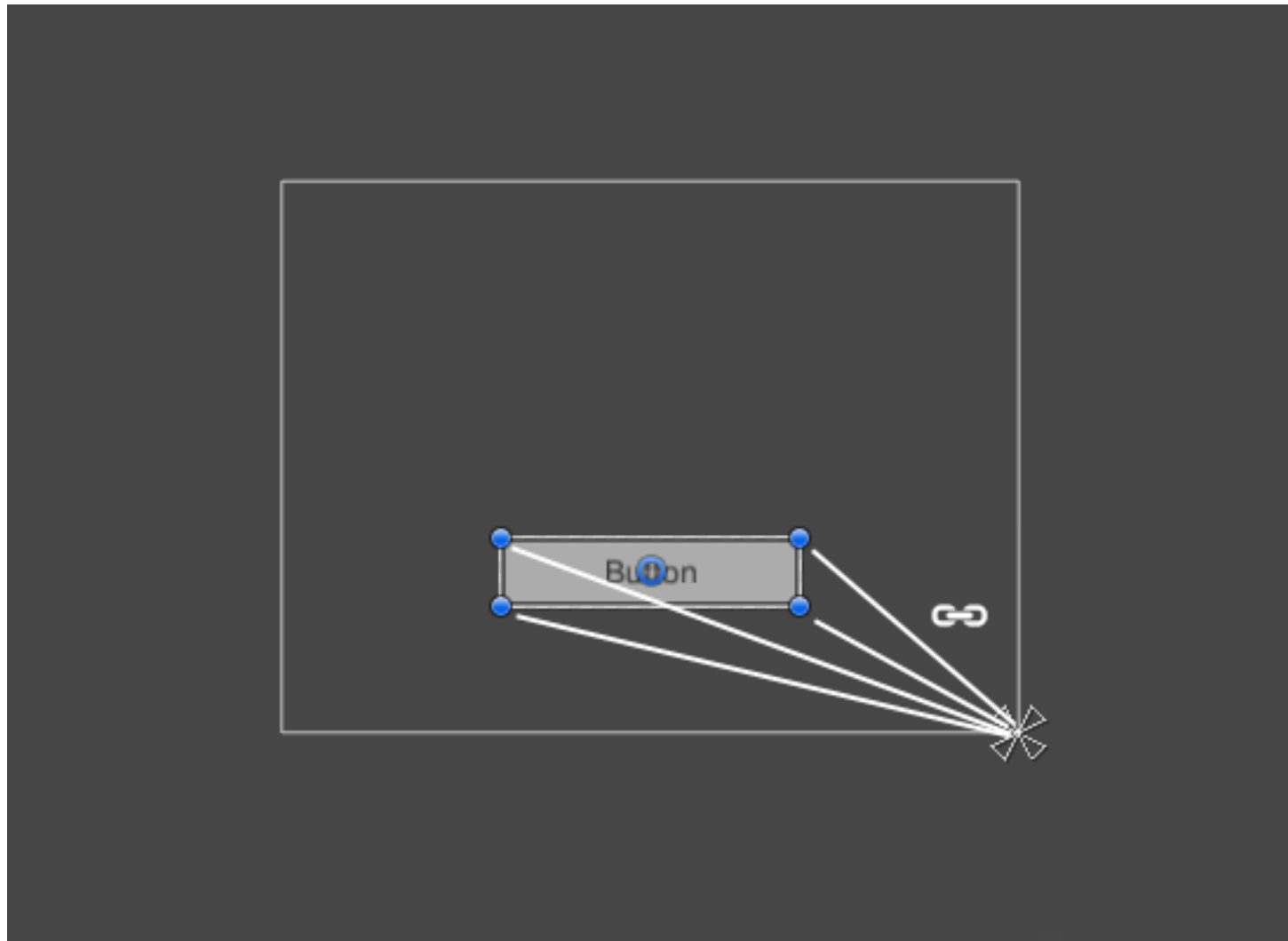


Anchors

- **Anchors** define how a UI element's **position and size** behave relative to its **parent's RectTransform**.
- In the **Scene View**, anchors appear as **four small triangular handles**, and their exact values can also be adjusted in the **Inspector**.
- If the parent object is also a RectTransform, the child can be **anchored to specific edges or regions** of the parent (e.g., center, corners, sides).
- Anchors can:
 - **Fix** the element's position relative to the parent (e.g., top-left, center).
 - Allow the element to **stretch** horizontally, vertically, or in both directions, as the parent's size changes.



Anchors



Anchors

- In the **Rect Transform** component (Inspector), you'll find the **Anchor Preset button** in the **top-left corner**. It provides quick access to a set of **commonly used anchoring layouts**.
- It is possible to anchor the UI element to the sides or middle of the parent or stretch together with the parent size.
- You can also fine-tune anchoring by editing the **Anchor Min** and **Anchor Max** values:
 - **Anchor Min** → defines the **lower-left corner** of the anchor region
 - **Anchor Max** → defines the **upper-right corner** of the anchor region
- These values are **normalized** (from 0 to 1), relative to the parent RectTransform's dimensions.



TextMesh Pro

- **TextMesh Pro** is an advanced text rendering system in Unity, replace the legacy Text component and provide **high-quality, flexible, and efficient** text display.
- Key features are:
 - **Crisp, resolution-independent text rendering**
 - Full support for **rich text tags** (bold, italics, color, size, etc.)
 - **Advanced formatting** options:
 - Outlines, shadows, gradients
 - Character, word, and line spacing
 - Kerning and font weight adjustments
 - Support for **dynamic and static font assets**
 - Works with both **UI (Canvas)** and **3D world-space text**

TextMesh Pro

- The TextMesh Pro package is included in the Unity Editor
- To use TextMesh Pro in your projects, you need to import the **TMP Essential Resources**.
- From the menu, select **Window > TextMeshPro > Import TMP Essential Resources**.
- To work with TextMesh Pro components in scripting you must import the UnityEngine.UI and TMPro libraries.

```
using UnityEngine.UI;  
using TMPro;
```

Visual Components

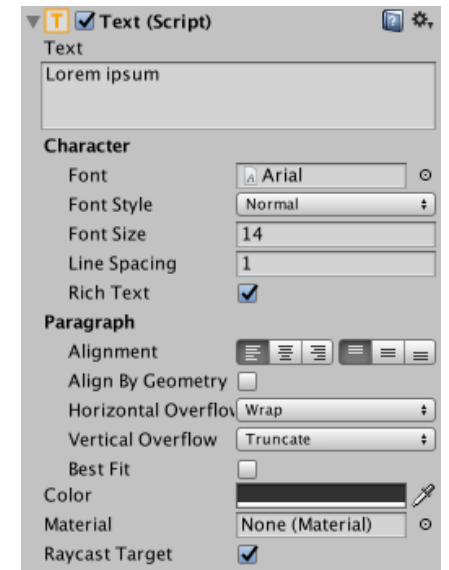
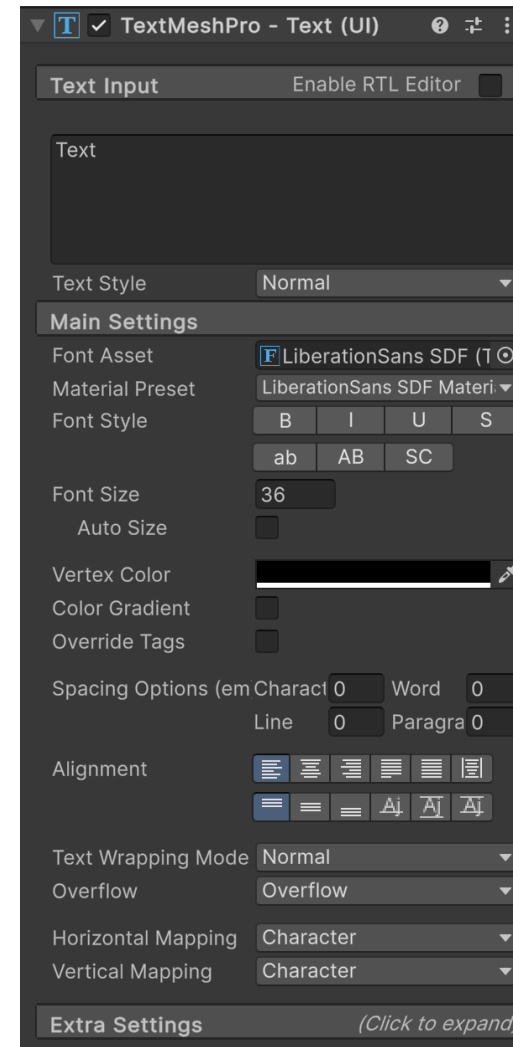
- With the introduction of the **UI system in Unity**, several specialized **components** were added to support the creation of **graphical user interfaces (GUI)** directly within the scene.
- **These components provide:**
 - Built-in support for **rendering UI elements** such as **images, text, buttons, sliders, and more**
 - Tools to control **layout, user interaction, and visual styling** within the Canvas
 - Seamless integration with the **event system** for handling user input

Visual Components: Text

- The **Text** component (also called **Label**) is used to display **basic text** in the UI.
- It is possible to set properties such as font, font style, font size, colour, etc.
- The Text component belongs to the **UnityEngine.UI** namespace.

```
using UnityEngine.UI;
```

```
Text testo;  
label.testo = "Text set via script";
```

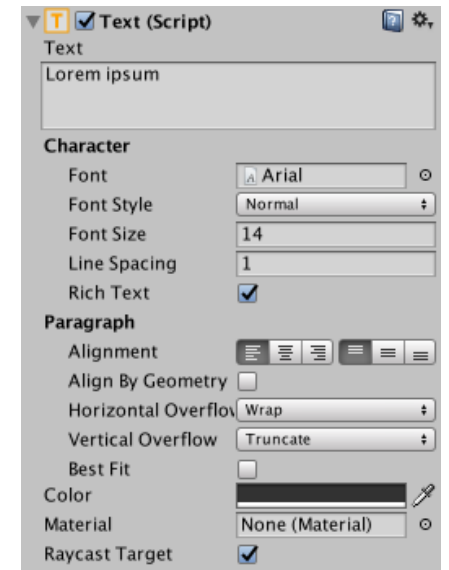
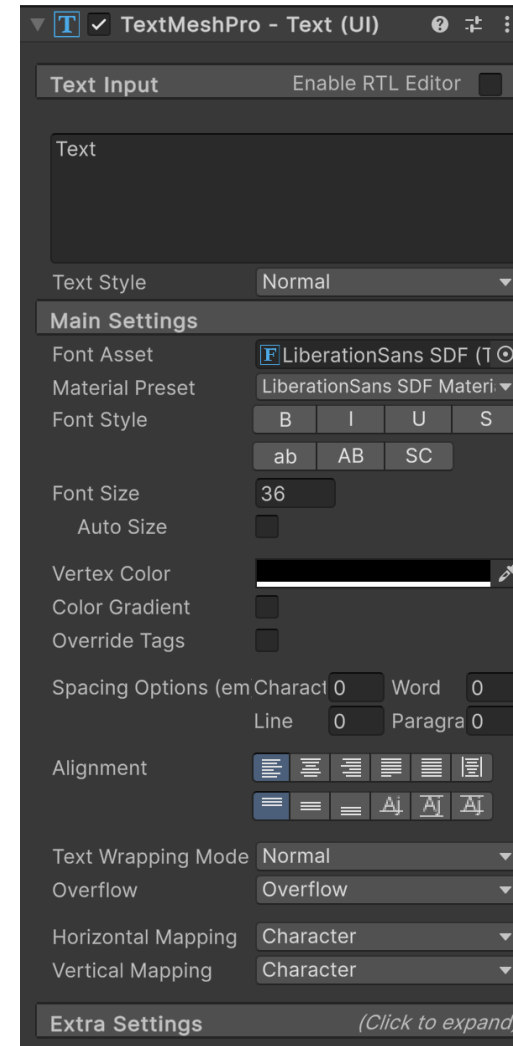


Visual Components: Text Mesh Pro

- TextMeshProUGUI is the **TextMesh Pro version** of the UI Text component.
- Offers **higher visual quality** and **advanced formatting options**, including, rich text tags (colour, size, bold, etc.), outlines, shadows, gradients and precise control over spacing and alignment.

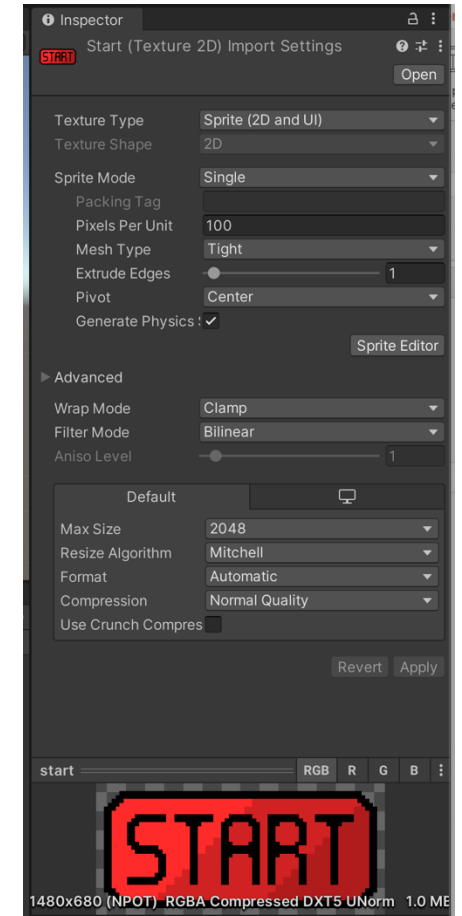
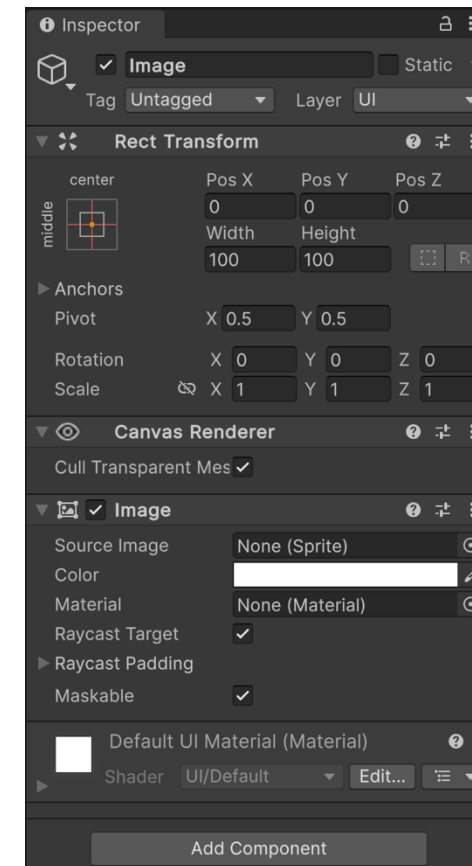
```
using UnityEngine.UI;  
using TMPro;
```

```
TextMeshProUGUI testo;  
testo.text = "Text set using TextMesh  
Pro";
```



Images and Sprites

- The **Image** UI element is made of:
 - **Rect Transform** for layout and positioning
 - **Image component** for rendering graphics
- In the **Image component**, assign a **Sprite** via the **Target Graphic** field.
- Images can be imported as **UI sprites** by selecting **Sprite(2D/UI)** from the 'Texture Type' settings.
- A material can also be applied to the Image component.

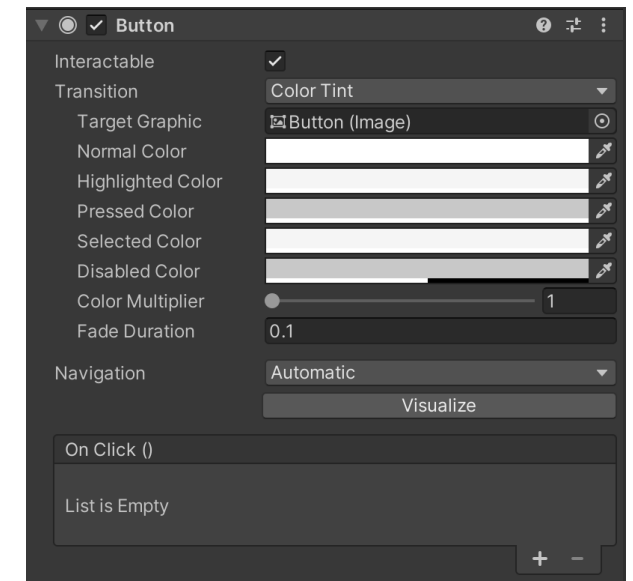


Images and Sprites

- The **Image Type** property controls how the **assigned Sprite** is rendered and resized within the Image component:
 - **Simple** - Scales the whole sprite equally.
 - **Sliced** - Utilises the 3x3 sprite division so that resizing does not distort corners and only the center part is stretched.
 - **Tiled** - Similar to Sliced, but tiles (repeats) the center part rather than stretching it. For sprites with no borders at all, the entire sprite is tiled.
 - **Filled** - Shows the sprite in the same way as Simple does except that it fills in the sprite from an origin in a defined direction, method and amount.
- The option to Set Native Size, available with Simple or Filled is selected, resets the image to to the **original dimensions** of the sprite.

Button

- A **Button** is a UI element that **responds to user clicks** and is commonly used to **trigger or confirm actions** in the interface.
- To create a button: **GameObject -> UI -> Button.**
- **Key properties:**
 - **Interactable:** enable if you want this button to accept input
 - **Transition:** controls the **visual response** of the button to user interaction (e.g., hover, click, press).
 - **Navigation:** defines how the button behaves with **keyboard/controller navigation** (e.g., Tab, arrows).
- The button has a single event called **OnClick()** that responds when the user completes a click.



Button: example 1



- Attach the script to a GameObject (e.g., an empty GameObject).
- Assign the Button component to myButton in the **Inspector**.
- The method `OnButtonClick()` will be called when the button is pressed.

```
using UnityEngine.UI;

public class ButtonExample : MonoBehaviour{

    public Button myButton;

    void Start(){

        // Register the event listener

        myButton.onClick.AddListener(OnButtonClick);

    }

    void OnButtonClick(){

        // Action to perform when the button is clicked

        Debug.Log("Button was clicked!");

    }

}
```

Button: example 2



- **myButton:** assign the Button component from inspector
- **buttonLabel:** assign the TextMeshProUGUI component inside the Button (the label).
- **messageLabel:** assign a separate TextMeshProUGUI text field (e.g., under a panel or UI canvas).

```
using UnityEngine;
using UnityEngine.UI;
using TMPro;

public class TMPButtonExample : MonoBehaviour
{
    public Button myButton;          // The UI Button
    public TextMeshProUGUI buttonLabel; // TMP button label
    public TextMeshProUGUI messageLabel; //TMP label for
    output

    void Start() {
        // Register the click event
        myButton.onClick.AddListener(OnButtonClicked);
    }

    void OnButtonClicked()
    {
        // Update the button label and the output message
        buttonLabel.text = "Clicked!";
        messageLabel.text = "You clicked the TMP button!";
    }
}
```



Esercizio UI

- Modificare lo sfondo dell'esercizio 3 aggiungendo uno skybox differente.
- Modificare l'esercizio 3 di riepilogo aggiungendo due bottoni e una scritta all'interno della scena.
- Se la sfera cade nel vuoto dovrà apparire la scritta «Game Over!»
- Se, invece, la sfera colpisce il wall4 allora dovrà apparire la scritta «Win!» (inizialmente il testo è vuoto).
- Aggiungere un tasto che fa cambiare casualmente colore alla sfera.
- Aggiungere un tasto che fa cambiare colore a wall1 sulla base di tre valori RGB passati dall'utente in input nell'inspector.

Saving and Exporting a Scene

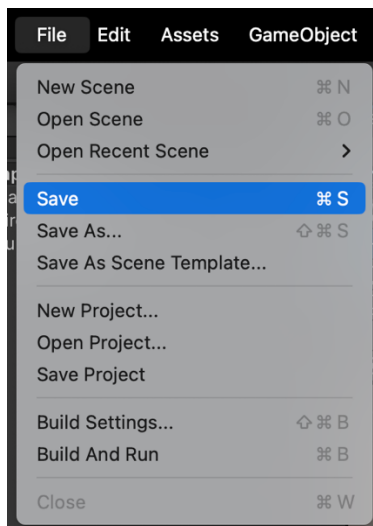


How to save

Unity stores different types of information about your project, and **not all changes are saved the same way**. It's important to understand the difference between **saving a scene** and **saving a project**.

✓ SAVE THE SCENE!

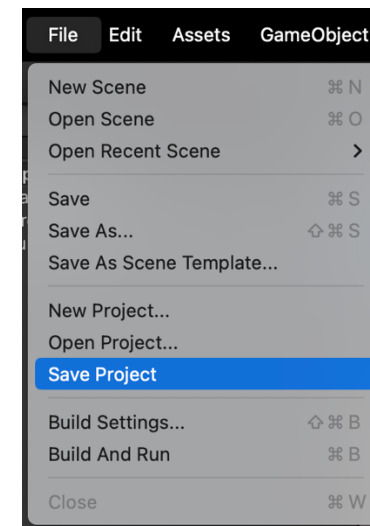
- Go to **File** → **Save** or press **Ctrl + S (Windows)** / **Cmd + S (Mac)**.
- This saves the current scene, including objects, lighting, and layout.



!! If you don't save the scene, any changes will be lost when Unity closes **!!**

✓ SAVE THE PROJECT!

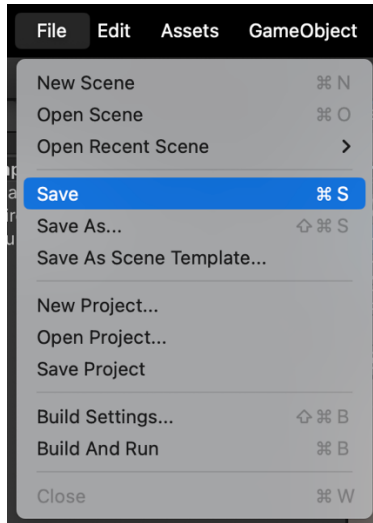
- Go to File → Save Project.
- This saves **project-wide settings**, such as imported assets, prefab modifications, and package settings.



How to save

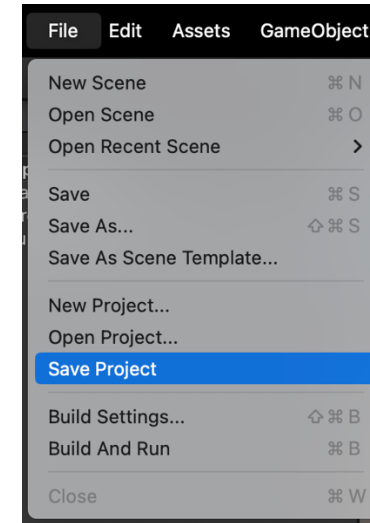
Unity stores different types of information about your project, and **not all changes are saved the same way**. It's important to understand the difference between **saving a scene** and **saving a project**.

✓ SAVE THE SCENE!



!! If you don't save the scene, any changes will be lost when Unity closes !!

✓ SAVE THE PROJECT!



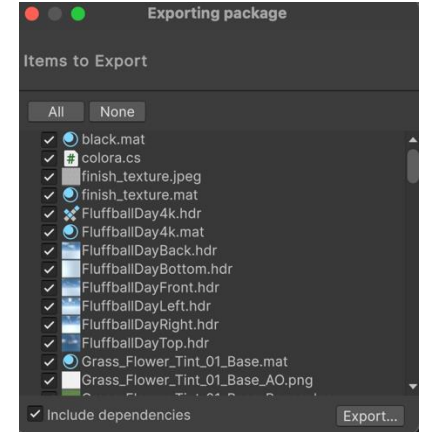
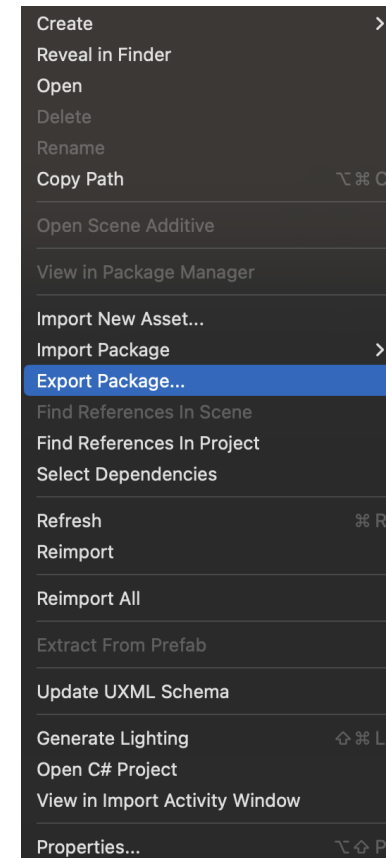
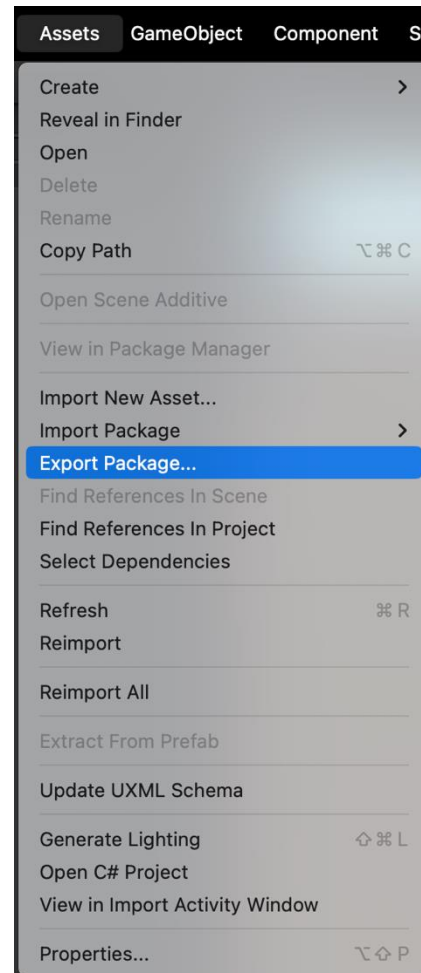
NOTE! Using “Save Project” **does not save** changes to your Scene, only the project-wide changes are saved. You may want to, for instance, save your project but not changes to your scene if you have used a temporary scene to make some changes to a prefab.

Project Export



If you want to transfer your assets into a different project and preserve all the project's information, you can export your **assets** as a **Custom Package**.

1. Open the project you want to export assets from.
2. Choose **Assets > Export Package...** from the menu to bring up the **Exporting Package** dialog box.
3. In the dialog box, select the assets you want to include in the package by clicking on the boxes so they are checked.
4. Leave the **include dependencies** box checked to auto-select any assets used by the ones you have selected.
5. Click on **Export** to bring up File Explorer (Windows) or Finder (Mac) and choose where you want to store your package file. Name and save the package anywhere you like.



Project Export



WARNING: When exporting the project, make sure you are in the **correct folder**. To export the entire Assets folder, you must be inside the Assets folder. If you select a subfolder, only the elements within that subfolder will be exported, and the rest will not be exported!!!

1. Open the project you want to export assets from.
2. Choose **Assets > Export Package...** from the menu to bring up the **Exporting Package** dialog box.
3. In the dialog box, select the assets you want to include in the package by clicking on the boxes so they are checked.
4. Leave the **include dependencies** box checked to auto-select any assets used by the ones you have selected.
5. Click on **Export** to bring up File Explorer (Windows) or Finder (Mac) and choose where you want to store your package file. Name and save the package anywhere you like.

